

Updated_Q1_Q2_Q3_Analysis (corrected + crisis robustness)

Promi Roy

2026-03-19

```
# Global figure sizing. A wider default device stops long ggplot titles  
# (e.g. the 30-day volatility and DCC correlation charts) from being clipped,  
# and 300 dpi keeps every figure legible in the printed PDF.
```

```
knitr::opts_chunk$set(fig.width = 8, fig.height = 4.8, dpi = 300,  
  fig.align = "center", out.width = "95%")
```

```
library(forecast)  
library(tseries)  
library(ggplot2)  
library(lmtest)  
library(FinTS)  
library(scales)  
library(zoo)  
library(urca)  
library(vars)  
library(tsDyn)  
library(reshape2)  
library(corrplot)  
library(knitr)  
library(rugarch)  
library(rmgarch)  
library(aod)  # wald.test, for Toda-Yamamoto Granger causality  
library(car)  # LeveneTest, robust equal-variance test  
library(dplyr)
```

```
# ---- Shared ggplot theme for print-readable figures -----  
# Enlarges axis labels, titles, and legend text and trims white space, so the  
# figures stay legible in the printed PDF. Applied to every ggplot below in  
# place of the earlier theme_minimal() calls.
```

```
theme_thesis <- function(base = 14) {  
  theme_minimal(base_size = base) +  
  theme(  
    plot.title       = element_text(size = base + 2),  
    plot.subtitle    = element_text(size = base - 2, colour = "grey30"),  
    axis.title       = element_text(size = base),  
    axis.text        = element_text(size = base - 2),  
    legend.text      = element_text(size = base - 1),  
    legend.title     = element_text(size = base - 1),  
    legend.position  = "bottom",  
    legend.key.width = unit(1.4, "lines"),  
    panel.grid.minor = element_blank(),
```

```

    plot.margin      = margin(4, 8, 4, 4)
  )
}

# =====
# Data Preparation
# =====
# NOTE (fix #2): we do NOT overwrite the source CSV any more.
# The raw file is read once and never written back, so the
# pipeline is reproducible: running it twice from the same raw
# file always gives the same result. Any derived columns stay
# in memory (and, if needed, are written to a NEW file).
#
# NOTE (fix #1): the old code built a backward-looking
# "VIX_proxy_daily" (30-day rolling SD of S&P 500 returns) and
# used it for the stress/calm split. The thesis explicitly says
# it uses the official, option-implied CBOE VIX rather than a
# rolling-volatility proxy, so that proxy contradicts the text.
# We rely on the real VIX_CBOE column from the dataset instead
# and have removed the proxy construction entirely.
# =====

raw_path <- "C:/Users/Lenovo/Downloads/Gold_Data_Final_clean.csv" # <--
edit path if needed

gold_data <- read.csv(raw_path, header = TRUE)
colnames(gold_data)[1] <- "Date"
gold_data$Date <- as.Date(gold_data$Date, format = "%m/%d/%Y")

# Rename columns to short names used throughout this script
colnames(gold_data)[colnames(gold_data) == "Gold_XAUUSD"] <- "Gold"
colnames(gold_data)[colnames(gold_data) == "USD_Index_DXY"] <-
"USD_Index"
colnames(gold_data)[colnames(gold_data) == "SP500_.GSPC"] <- "SP500"
colnames(gold_data)[colnames(gold_data) == "Inflation_CPI"] <-
"Inflation_CPI"
colnames(gold_data)[colnames(gold_data) == "X10Y_Treasury_Yield"] <-
"X10Y_Treasury_Yield"
colnames(gold_data)[colnames(gold_data) == "Unemployment_Rate"] <-
"Unemployment"
colnames(gold_data)[colnames(gold_data) == "Bitcoin_BTC.USD"] <-
"Bitcoin"
colnames(gold_data)[colnames(gold_data) == "Silver_XAGUSD"] <- "Silver"
colnames(gold_data)[colnames(gold_data) == "Crude_Oil_WTI_CL.F"] <-
"Crude_Oil"

# Confirm the official VIX column is present (used later for the
# regime split). The dataset documents it as VIX_CBOE (from FRED).
if (!"VIX_CBOE" %in% colnames(gold_data)) {

```

```

    warning("VIX_CBOE column not found. Check the column name in the raw CSV;
",
           "the regime split in RQ3 requires the official CBOE VIX series.")
}

cat("Renamed columns:\n")

## Renamed columns:

print(colnames(gold_data))

##   [1] "Date"          "Gold"          "USD_Index"
##   [4] "Crude_Oil"     "Silver"        "SP500"
##   [7] "Bitcoin"       "Interest_Rate" "X10Y_Treasury_Yield"
##  [10] "Inflation_CPI" "Unemployment"  "VIX_CBOE"

gold_data <- gold_data[!is.na(gold_data$Gold), ]

# Guard (data fix, v7): the original CSV contained 74 placeholder rows
# on calendar month-starts (weekends/holidays) with Gold, SP500 and
# the USD index carried forward from the prior trading day. The
# cleaned CSV removes them. These checks fail loudly if a stale file
# is ever used again.
stopifnot(!any(format(gold_data$Date, "%u") %in% c("6", "7")))
stale_chk <- with(gold_data,
                  Gold == dplyr::lag(Gold) &
                  SP500 == dplyr::lag(SP500) &
                  USD_Index == dplyr::lag(USD_Index))
stopifnot(sum(stale_chk, na.rm = TRUE) == 0)

cat("Observations:", nrow(gold_data), "| Range:",
    as.character(min(gold_data$Date)), "to",
    as.character(max(gold_data$Date)), "\n")

## Observations: 4443 | Range: 2008-01-02 to 2025-08-29

print(summary(gold_data$Gold))

##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   704.9 1222.5 1364.4 1534.5 1784.5 3473.7

# RQ1: Can historical gold prices forecast future gold prices?

gold_zoo <- zoo(gold_data$Gold, order.by = gold_data$Date)
gold_ts <- ts(gold_data$Gold, frequency = 252)

n <- length(gold_ts)
h <- 60

# Train-Test Split

```

```

train_ts    <- window(gold_ts, end = c(1, n - h))
test_ts     <- window(gold_ts, start = c(1, n - h + 1))
train_dates <- gold_data$Date[1:(n - h)]
test_dates  <- gold_data$Date[(n - h + 1):n]

ggplot(gold_data, aes(x = Date, y = Gold)) +
  geom_line(color = "#DAA520", linewidth = 0.5) +
  geom_vline(xintercept = as.numeric(test_dates[1]),
             linetype = "dashed", color = "red", linewidth = 0.6) +
  labs(title = "Gold Price (USD/oz): 2008-2025",
       x = "Date", y = "Price (USD)") +
  theme_thesis()

```



```

# Stationarity test
# NOTE (small fix): tseries::adf.test and kpss.test clamp the
# reported p-value to a printed range (e.g. 0.01 / 0.99) and warn
# "p-value smaller/greater than printed". Report these as bounds
# ("< 0.01", "> 0.99"), not as exact figures.

adf_level <- adf.test(train_ts, alternative = "stationary")
kpss_level <- kpss.test(train_ts, null = "Level")

gold_diff <- diff(train_ts)
adf_diff <- adf.test(gold_diff, alternative = "stationary")
kpss_diff <- kpss.test(gold_diff, null = "Level")

ndiffs_result <- ndiffs(train_ts, test = "adf")

# Helper to display clamped p-values as bounds

```

```

fmt_p <- function(p) {
  if (p <= 0.01) return("< 0.01")
  if (p >= 0.99) return("> 0.99")
  sprintf("%.4f", p)
}

cat("\nStationarity (p-values clamped by tseries are shown as bounds):\n")

##
## Stationarity (p-values clamped by tseries are shown as bounds):

cat("  Level -> ADF p:", fmt_p(adf_level$p.value),
    "| KPSS p:", fmt_p(kpss_level$p.value), "\n")

##   Level -> ADF p: > 0.99 | KPSS p: < 0.01

cat("  Diff'd -> ADF p:", fmt_p(adf_diff$p.value),
    "| KPSS p:", fmt_p(kpss_diff$p.value), "\n")

##   Diff'd -> ADF p: < 0.01 | KPSS p: 0.0358

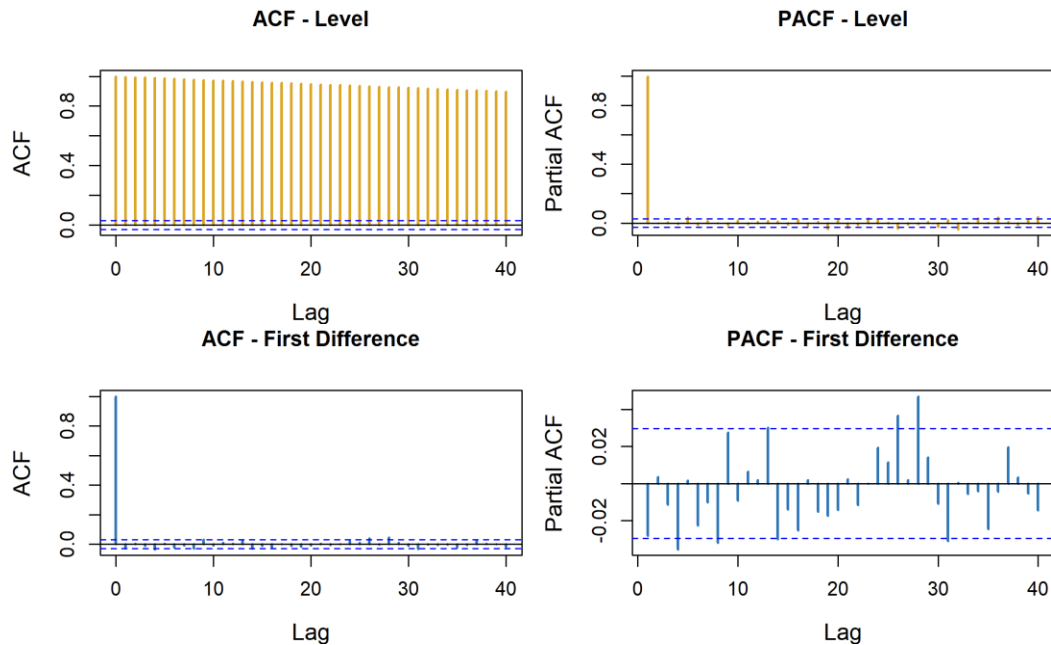
cat("  Integration order: I(", ndiffs_result, ")\n")

##   Integration order: I( 1 )

# ACF / PACF Plots

par(mfrow = c(2, 2), mar = c(4, 4, 3, 1), cex.lab = 1.25, cex.axis = 1.05,
    cex.main = 1.15)
acf(as.numeric(train_ts), lag.max = 40, main = "ACF - Level", col =
"#DAA520", lwd = 2)
pacf(as.numeric(train_ts), lag.max = 40, main = "PACF - Level", col =
"#DAA520", lwd = 2)
acf(as.numeric(gold_diff), lag.max = 40, main = "ACF - First Difference", col =
"#2E75B6", lwd = 2)
pacf(as.numeric(gold_diff), lag.max = 40, main = "PACF - First Difference",
col = "#2E75B6", lwd = 2)

```



ARIMA Model Selection

```
auto_model <- auto.arima(train_ts,
                          d = 1,
                          seasonal = FALSE,
                          stepwise = FALSE,
                          approximation = FALSE,
                          trace = TRUE,
                          ic = "aicc")
```

##		
##	ARIMA(0,1,0)	: 37368.94
##	ARIMA(0,1,0)	with drift : 37366.14
##	ARIMA(0,1,1)	: 37367.77
##	ARIMA(0,1,1)	with drift : 37364.69
##	ARIMA(0,1,2)	: 37369.66
##	ARIMA(0,1,2)	with drift : 37366.63
##	ARIMA(0,1,3)	: 37370.96
##	ARIMA(0,1,3)	with drift : 37367.78
##	ARIMA(0,1,4)	: 37367.73
##	ARIMA(0,1,4)	with drift : 37364.14
##	ARIMA(0,1,5)	: 37369.62
##	ARIMA(0,1,5)	with drift : 37366.08
##	ARIMA(1,1,0)	: 37367.74
##	ARIMA(1,1,0)	with drift : 37364.66
##	ARIMA(1,1,1)	: 37369.75
##	ARIMA(1,1,1)	with drift : 37366.68
##	ARIMA(1,1,2)	: 37371.66
##	ARIMA(1,1,2)	with drift : 37368.63
##	ARIMA(1,1,3)	: 37368.8
##	ARIMA(1,1,3)	with drift : 37364.54

```

## ARIMA(1,1,4) : Inf
## ARIMA(1,1,4) with drift : Inf
## ARIMA(2,1,0) : 37369.65
## ARIMA(2,1,0) with drift : 37366.62
## ARIMA(2,1,1) : 37371.66
## ARIMA(2,1,1) with drift : 37368.62
## ARIMA(2,1,2) : Inf
## ARIMA(2,1,2) with drift : Inf
## ARIMA(2,1,3) : 37370.62
## ARIMA(2,1,3) with drift : 37366.42
## ARIMA(3,1,0) : 37371.19
## ARIMA(3,1,0) with drift : 37368.05
## ARIMA(3,1,1) : 37368.97
## ARIMA(3,1,1) with drift : 37364.69
## ARIMA(3,1,2) : 37370.54
## ARIMA(3,1,2) with drift : 37366.63
## ARIMA(4,1,0) : 37368.02
## ARIMA(4,1,0) with drift : 37364.52
## ARIMA(4,1,1) : Inf
## ARIMA(4,1,1) with drift : Inf
## ARIMA(5,1,0) : 37369.99
## ARIMA(5,1,0) with drift : 37366.51
##
##
## Best model: ARIMA(0,1,4) with drift

print(summary(auto_model))

## Series: train_ts
## ARIMA(0,1,4) with drift
##
## Coefficients:
##          ma1      ma2      ma3      ma4      drift
##      -0.0283  0.0023  -0.0140  -0.0371  0.5690
## s.e.   0.0151  0.0151   0.0155   0.0156  0.2394
##
## sigma^2 = 295.1: log likelihood = -18676.06
## AIC=37364.12  AICc=37364.14  BIC=37402.43
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE
MASE
## Training set -8.994257e-05 17.16618 11.63207 -0.01718807 0.780779
0.05713062
##              ACF1
## Training set -0.0001324919

print(coeftest(auto_model))

```

```
##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
## ma1   -0.0282686  0.0151134 -1.8704  0.06142 .
## ma2    0.0023295  0.0151438  0.1538  0.87775
## ma3   -0.0139752  0.0154694 -0.9034  0.36631
## ma4   -0.0370880  0.0155831 -2.3800  0.01731 *
## drift  0.5690378  0.2393926  2.3770  0.01745 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Model comparison

```
candidates <- list(
  "ARIMA(0,1,0)" = Arima(train_ts, order = c(0, 1, 0)),
  "ARIMA(1,1,0)" = Arima(train_ts, order = c(1, 1, 0)),
  "ARIMA(0,1,1)" = Arima(train_ts, order = c(0, 1, 1)),
  "ARIMA(1,1,1)" = Arima(train_ts, order = c(1, 1, 1)),
  "ARIMA(2,1,0)" = Arima(train_ts, order = c(2, 1, 0)),
  "ARIMA(0,1,2)" = Arima(train_ts, order = c(0, 1, 2)),
  "ARIMA(2,1,1)" = Arima(train_ts, order = c(2, 1, 1)),
  "ARIMA(1,1,2)" = Arima(train_ts, order = c(1, 1, 2)),
  "ARIMA(2,1,2)" = Arima(train_ts, order = c(2, 1, 2)),
  "ARIMA(3,1,1)" = Arima(train_ts, order = c(3, 1, 1))
)
```

```
comparison <- data.frame(
  Model = names(candidates),
  AIC    = sapply(candidates, AIC),
  AICc   = sapply(candidates, function(m) m$aicc),
  BIC    = sapply(candidates, BIC)
)
comparison <- comparison[order(comparison$AICc), ]
rownames(comparison) <- NULL
print(comparison, digits = 4)
```

```
##      Model    AIC  AICc   BIC
## 1 ARIMA(1,1,0) 37368 37368 37381
## 2 ARIMA(0,1,1) 37368 37368 37381
## 3 ARIMA(0,1,0) 37369 37369 37375
## 4 ARIMA(3,1,1) 37369 37369 37401
## 5 ARIMA(2,1,0) 37370 37370 37389
## 6 ARIMA(0,1,2) 37370 37370 37389
## 7 ARIMA(1,1,1) 37370 37370 37389
## 8 ARIMA(2,1,1) 37372 37372 37397
## 9 ARIMA(1,1,2) 37372 37372 37397
## 10 ARIMA(2,1,2) 37374 37374 37406
```

```

# v8: ARIMA(0,1,0) is retained as the benchmark BY DESIGN -- it is
# the random-walk hypothesis under test in RQ1 (thesis Sections 5.2
# and 6.1). The candidate comparison and the auto.arima trace are
# reported for transparency, but the AICc spread across all models is
# negligible (<5 points over ~3,300 obs), so it does not override the
# random-walk choice. The comparison winner is printed only as a check.
aicc_winner <- comparison$Model[1]
best_name <- "ARIMA(0,1,0)"
best_model <- candidates[[best_name]]
cat("\n>>> BENCHMARK (by design): ARIMA(0,1,0) | lowest-AICc candidate
was:",
    aicc_winner, "<<<\n")

##
## >>> BENCHMARK (by design): ARIMA(0,1,0) | lowest-AICc candidate was:
ARIMA(1,1,0) <<<

cat(">>> AICc gap (benchmark - winner):",
    round(comparison$AICc[comparison$Model == "ARIMA(0,1,0)"] -
        comparison$AICc[comparison$Model == aicc_winner], 3),
    "points -- negligible, random walk retained. <<<\n")

## >>> AICc gap (benchmark - winner): 1.202 points -- negligible, random walk
retained. <<<

# Residual Diagnostics

residuals_best <- residuals(best_model)

for (lag in c(10, 20, 30)) {
  lb <- Box.test(residuals_best, lag = lag, type = "Ljung-Box",
                fitdf = sum(best_model$arma[1:2]))
  cat(sprintf("Ljung-Box lag %d: p = %.4f\n", lag, lb$p.value))
}

## Ljung-Box lag 10: p = 0.0259
## Ljung-Box lag 20: p = 0.0270
## Ljung-Box lag 30: p = 0.0053

arch_test <- ArchTest(residuals_best, lags = 12)
cat(sprintf("ARCH-LM: p = %s\n", format.pval(arch_test$p.value, digits = 4)))

## ARCH-LM: p = < 2.2e-16

sw_sample <- if (length(residuals_best) > 5000) {
  set.seed(42); sample(residuals_best, 5000)
} else residuals_best
sw_test <- shapiro.test(sw_sample)
cat(sprintf("Shapiro-Wilk: p = %s\n", format.pval(sw_test$p.value, digits =
4)))

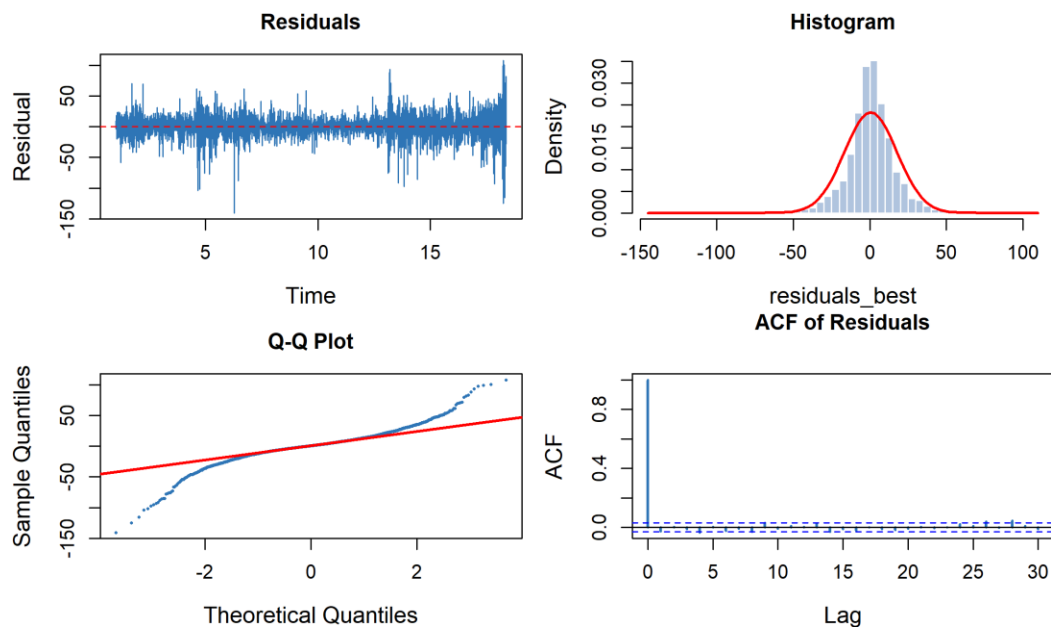
## Shapiro-Wilk: p = < 2.2e-16

```

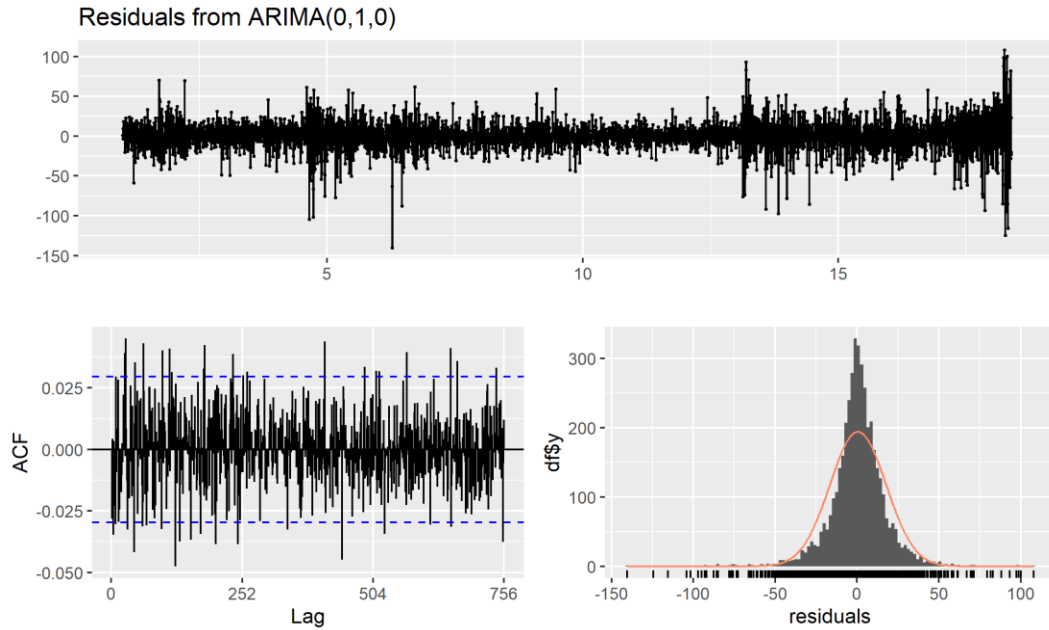
```

par(mfrow = c(2, 2), mar = c(4, 4, 3, 1), cex.lab = 1.25, cex.axis = 1.05,
    cex.main = 1.15)
plot(residuals_best, main = "Residuals", ylab = "Residual", col = "#2E75B6",
     type = "l")
abline(h = 0, col = "red", lty = 2)
hist(residuals_best, breaks = 50, prob = TRUE, main = "Histogram",
     col = "#B0C4DE", border = "white")
curve(dnorm(x, mean(residuals_best), sd(residuals_best)), add = TRUE, col =
"red", lwd = 2)
qqnorm(residuals_best, main = "Q-Q Plot", col = "#2E75B6", pch = 16, cex =
0.4)
qqline(residuals_best, col = "red", lwd = 2)
acf(as.numeric(residuals_best), lag.max = 30, main = "ACF of Residuals", col
= "#2E75B6", lwd = 2)

```



```
checkresiduals(best_model)
```



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,0)
## Q* = 573.35, df = 504, p-value = 0.01733
##
## Model df: 0.   Total lags used: 504

# Forecast 60 days

fc <- forecast(best_model, h = h, level = c(80, 95))

actual    <- as.numeric(test_ts)
predicted <- as.numeric(fc$mean)
errors    <- actual - predicted
mae       <- mean(abs(errors))
rmse      <- sqrt(mean(errors^2))
mape      <- mean(abs(errors / actual)) * 100

accuracy_results <- accuracy(fc, test_ts)
print(accuracy_results)

##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.5691437 17.19524 11.65561 0.02492295 0.7812610 0.05724624
## Test set     3.6833742 40.37708 31.80667 0.09556422 0.9444733 0.15621763
##              ACF1 Theil's U
## Training set -0.02816023      NA
## Test set     0.66190374  1.385258

# Benchmark comparison
# NOTE (fix #5): when the selected model is ARIMA(0,1,0), the
```

```

# fixed-origin ARIMA forecast is identical to the naive
# (random-walk) forecast, so a Diebold-Mariano test of ARIMA vs
# naive compares the model against itself and returns NA. That is
# not a meaningful test; the honest statement is simply that the
# random walk IS the naive benchmark.
#
# To get a genuine forecast comparison we instead test the chosen
# model against a DIFFERENT forecaster: ARIMA(0,1,0) WITH drift.
# The drift model is a real alternative (it adds a deterministic
# trend term), so the DM test is well defined.

drift_model <- Arima(train_ts, order = c(0, 1, 0), include.drift = TRUE)
drift_fc    <- forecast(drift_model, h = h)
drift_pred  <- as.numeric(drift_fc$mean)

e_best <- actual - predicted
e_drift <- actual - drift_pred

if (all(abs(e_best - e_drift) < 1e-8)) {
  cat("Chosen model and drift model produce identical forecasts; DM test not
applicable.\n")
  dm_pvalue <- NA
} else {
  dm_test2 <- dm.test(e_best, e_drift, alternative = "two.sided")
  dm_pvalue <- dm_test2$p.value
  cat(sprintf("\nDM test (%s vs ARIMA(0,1,0)+drift): stat = %.4f, p =
%.4f\n",
              best_name, dm_test2$statistic, dm_pvalue))
  cat("Interpretation: a non-significant p-value means the random walk\n")
  cat("is not outperformed by the drift alternative over this window.\n")
}

##
## DM test (ARIMA(0,1,0) vs ARIMA(0,1,0)+drift): stat = -0.2780, p = 0.7820
## Interpretation: a non-significant p-value means the random walk
## is not outperformed by the drift alternative over this window.

# For completeness, also report naive-benchmark accuracy. When the
# best model is ARIMA(0,1,0) these numbers will match the ARIMA
# fixed-origin numbers by construction, which is itself the point.
naive_fc <- naive(train_ts, h = h)
naive_acc <- accuracy(naive_fc, test_ts)
cat("\nNaive (random-walk) benchmark accuracy:\n")

##
## Naive (random-walk) benchmark accuracy:

print(naive_acc)

##
## Training set ME RMSE MAE MPE MAPE MASE
## Training set 0.569078 17.19720 11.65808 0.02490582 0.7814165 0.05725834

```

```
## Test set      3.683374 40.37708 31.80667 0.09556422 0.9444733 0.15621763
##              ACF1 Theil's U
## Training set -0.02816219      NA
## Test set     0.66190374 1.385258
```

Forecast Plots

NOTE (small fix): removed the stray double comma in geom_vline.

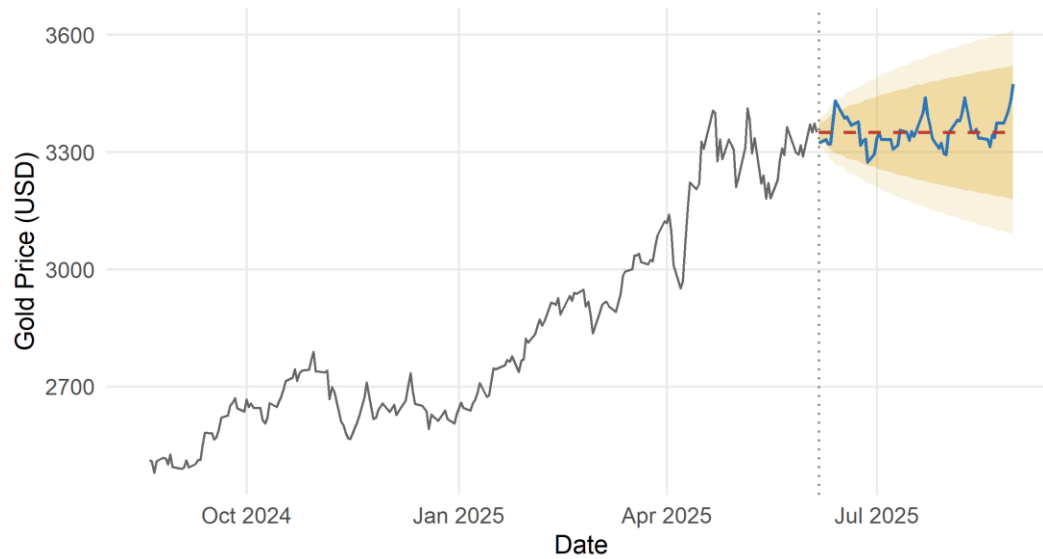
```
fc_df <- data.frame(
  Date = test_dates, Actual = actual, Forecast = predicted,
  Lo80 = as.numeric(fc$lower[, 1]), Hi80 = as.numeric(fc$upper[, 1]),
  Lo95 = as.numeric(fc$lower[, 2]), Hi95 = as.numeric(fc$upper[, 2])
)

recent_train <- data.frame(
  Date = tail(train_dates, 200),
  Gold = tail(as.numeric(train_ts), 200)
)

ggplot() +
  geom_ribbon(data = fc_df, aes(x = Date, ymin = Lo95, ymax = Hi95),
    fill = "#DAA520", alpha = 0.15) +
  geom_ribbon(data = fc_df, aes(x = Date, ymin = Lo80, ymax = Hi80),
    fill = "#DAA520", alpha = 0.3) +
  geom_line(data = recent_train, aes(x = Date, y = Gold), color = "gray40",
    linewidth = 0.6) +
  geom_line(data = fc_df, aes(x = Date, y = Actual), color = "#2E75B6",
    linewidth = 0.8) +
  geom_line(data = fc_df, aes(x = Date, y = Forecast), color = "#C0392B",
    linewidth = 0.8, linetype = "dashed") +
  geom_vline(xintercept = test_dates[1], linetype = "dotted", color =
    "gray50") +
  labs(title = paste("ARIMA Forecast vs Actual -", best_name),
    subtitle = paste0("RMSE = $", round(rmse, 2), " | MAPE = ",
    round(mape, 2), "%"),
    x = "Date", y = "Gold Price (USD)") +
  theme_thesis()
```

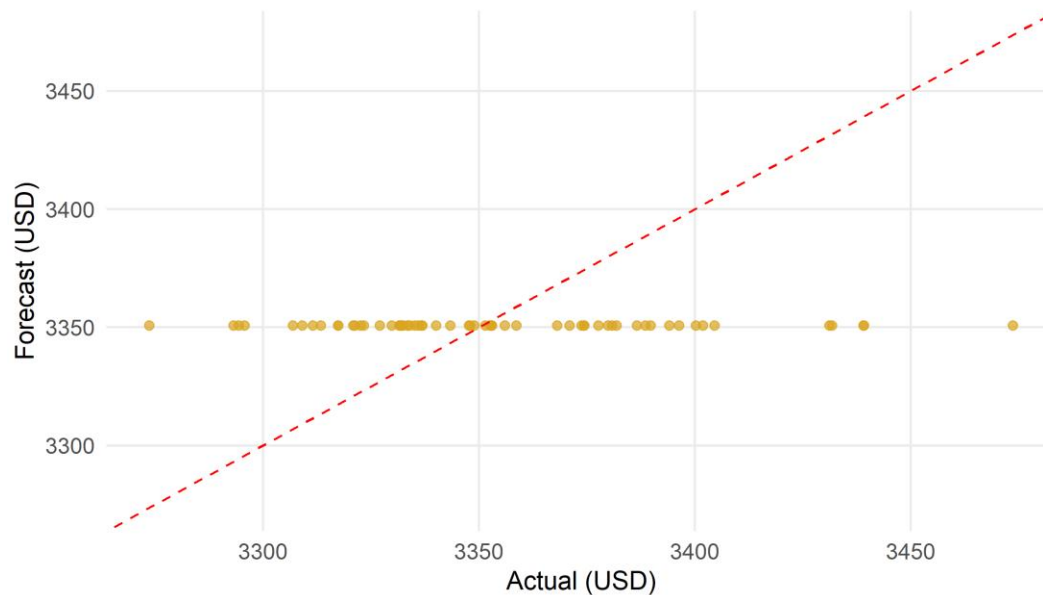
ARIMA Forecast vs Actual - ARIMA(0,1,0)

RMSE = \$40.38 | MAPE = 0.94%



```
ggplot(fc_df, aes(x = Actual, y = Forecast)) +  
  geom_point(color = "#DAA520", size = 2, alpha = 0.7) +  
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +  
  labs(title = "Actual vs Predicted", x = "Actual (USD)", y = "Forecast  
(USD)") +  
  theme_thesis() +  
  ylim(min(fc_df$Actual), max(fc_df$Actual))
```

Actual vs Predicted



Rolling 1 step ahead forecast

```
rolling_preds <- numeric(h)
```

```

rolling_actual <- numeric(h)

order_p <- best_model$arma[1]
order_q <- best_model$arma[2]
order_d <- best_model$arma[6]

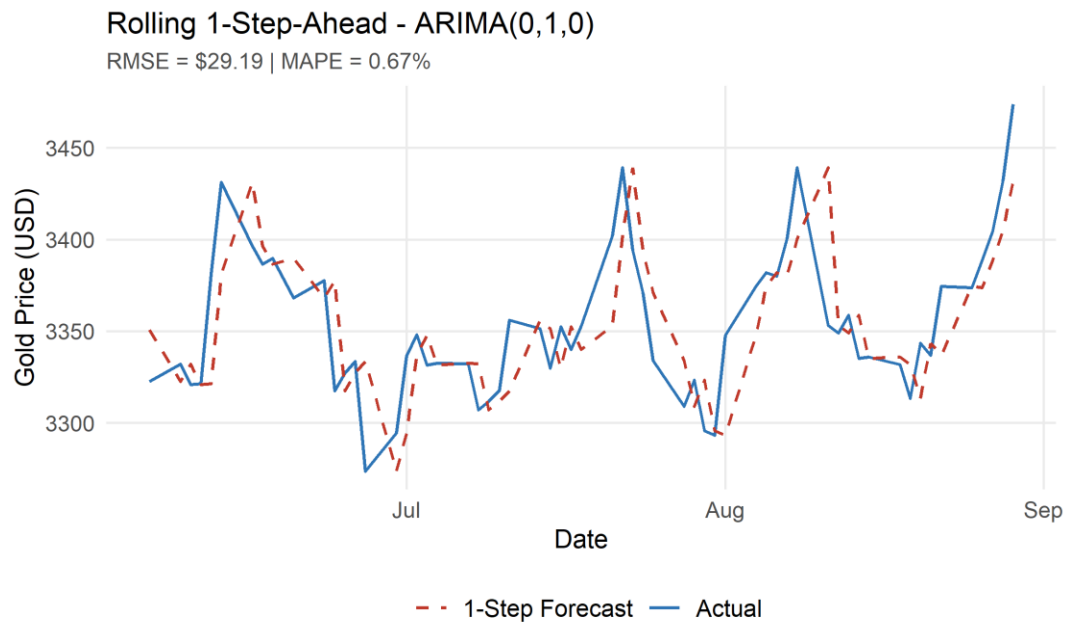
for (i in 1:h) {
  train_window <- gold_ts[1:(n - h + i - 1)]
  refit <- Arima(train_window, order = c(order_p, order_d, order_q))
  rolling_preds[i] <- as.numeric(forecast(refit, h = 1)$mean)
  rolling_actual[i] <- gold_ts[n - h + i]
}

rolling_errors <- rolling_actual - rolling_preds
rolling_rmse <- sqrt(mean(rolling_errors^2))
rolling_mae <- mean(abs(rolling_errors))
rolling_mape <- mean(abs(rolling_errors / rolling_actual)) * 100

rolling_df <- data.frame(Date = test_dates, Actual = rolling_actual, Forecast
= rolling_preds)

ggplot(rolling_df) +
  geom_line(aes(x = Date, y = Actual, color = "Actual"), linewidth = 0.8) +
  geom_line(aes(x = Date, y = Forecast, color = "1-Step Forecast"),
            linewidth = 0.8, linetype = "dashed") +
  scale_color_manual(values = c("Actual" = "#2E75B6", "1-Step Forecast" =
"#C0392B")) +
  labs(title = paste0("Rolling 1-Step-Ahead - ", best_name),
       subtitle = paste0("RMSE = $", round(rolling_rmse, 2),
                          " | MAPE = ", round(rolling_mape, 2), "%"),
       x = "Date", y = "Gold Price (USD)", color = "") +
  theme_thesis() + theme(legend.position = "bottom")

```



```
# NOTE (small fix): the rolling-summary cat() was printed twice in
# the original; duplicate removed.
cat("Best model:", best_name, "\n")

## Best model: ARIMA(0,1,0)

cat("Integration order: I(", ndiffs_result, ")\n")

## Integration order: I( 1 )

cat(sprintf("Fixed-origin -> RMSE=%.2f MAE=%.2f MAPE=%.2f%%\n", rmse,
mae, mape))

## Fixed-origin -> RMSE=$40.38 MAE=$31.81 MAPE=0.94%

cat(sprintf("Rolling 1-step -> RMSE=%.2f MAE=%.2f MAPE=%.2f%%\n",
rolling_rmse, rolling_mae, rolling_mape))

## Rolling 1-step -> RMSE=$29.19 MAE=$22.44 MAPE=0.67%

# RQ2: Do macroeconomic variables influence gold prices?

vars_rq2 <- gold_data[, c("Date", "Gold", "USD_Index", "SP500",
" Inflation_CPI", "X10Y_Treasury_Yield",
" Unemployment")]
colnames(vars_rq2)[6] <- "Treasury_10Y"

vars_rq2 <- vars_rq2[complete.cases(vars_rq2), ]
cat("Observations:", nrow(vars_rq2), "\n")

## Observations: 4443
```

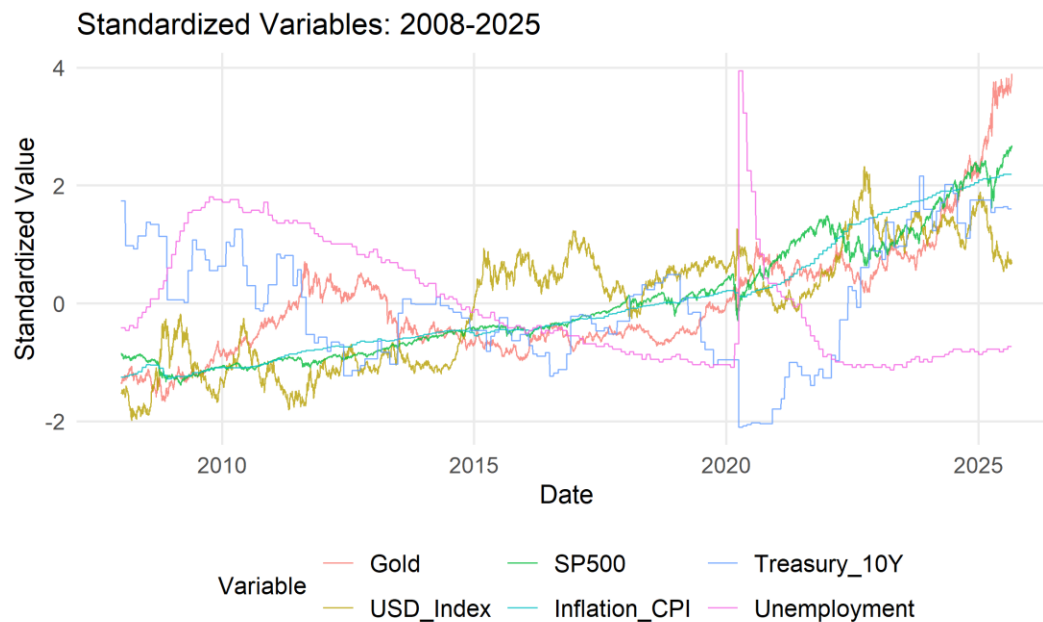
```

vars_numeric <- vars_rq2[, -1]
vars_scaled <- as.data.frame(scale(vars_numeric))
vars_scaled$Date <- vars_rq2$Date

melted <- melt(vars_scaled, id.vars = "Date", variable.name = "Variable")

ggplot(melted, aes(x = Date, y = value, color = Variable)) +
  geom_line(linewidth = 0.4, alpha = 0.8) +
  labs(title = "Standardized Variables: 2008-2025",
       x = "Date", y = "Standardized Value") +
  theme_thesis() +
  theme(legend.position = "bottom")

```



```

# Correlation Matrix
# NOTE (small fix): this block and its print() were duplicated in
# the original RQ3 section; kept once here where it belongs.

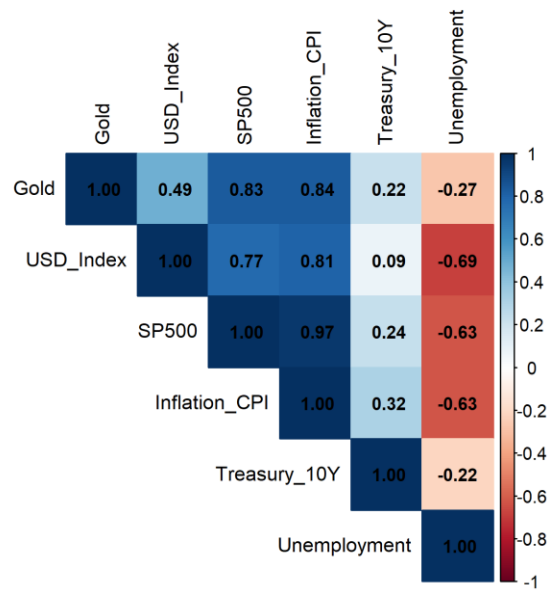
```

```

cor_matrix <- cor(vars_numeric, use = "complete.obs")
corrplot(cor_matrix, method = "color", type = "upper",
         addCoef.col = "black", number.cex = 0.8,
         tl.col = "black", tl.cex = 0.9,
         title = "Correlation Matrix", mar = c(0, 0, 2, 0))

```

Correlation Matrix



```
print(round(cor_matrix, 3))
```

```
##           Gold USD_Index  SP500 Inflation_CPI Treasury_10Y
Unemployment
## Gold      1.000      0.489  0.833           0.836      0.222      -
0.275
## USD_Index  0.489      1.000  0.774           0.807      0.089      -
0.687
## SP500      0.833      0.774  1.000           0.970      0.238      -
0.625
## Inflation_CPI 0.836      0.807  0.970           1.000      0.317      -
0.627
## Treasury_10Y 0.222      0.089  0.238           0.317      1.000      -
0.216
## Unemployment -0.275     -0.687 -0.625           -0.627     -0.216
1.000
```

```
# NOTE (v7): the daily-frequency VECM / Johansen / IRF / FEVD /
# Granger / CUSUM blocks were removed. The thesis reports the MONTHLY
# system only (Sections 5.3 and 6.2); the daily system mixed forward-
# filled monthly macro series into daily rows, which the thesis itself
# identifies as invalid (Sections 1.4, 2.7, 4.5). ALL RQ2 results now
# come exclusively from the monthly chunks below.
```

```
# =====
# RQ2 MAIN ANALYSIS: VECM AT MONTHLY FREQUENCY
# =====
# Estimates the VECM at the natural (monthly) frequency of the
# macro variables. This is the system reported in the thesis.
# In the daily data CPI, the 10-year yield and unemployment are
# monthly series forward-filled to daily; treating each daily row
```

```

# as independent inflates their effective sample size. Sampling at
# month-end puts all six variables on a common monthly grid.
# =====

# Aggregate to monthly by keeping the last trading day of each
# calendar month (end-of-month sampling for the daily market
# variables; recovers the underlying monthly macro values).

gold_monthly <- gold_data %>%
  mutate(YearMonth = format(Date, "%Y-%m")) %>%
  group_by(YearMonth) %>%
  slice_tail(n = 1) %>%
  ungroup() %>%
  as.data.frame()

cat("Daily observations: ", nrow(gold_data), "\n")
## Daily observations: 4443

cat("Monthly observations:", nrow(gold_monthly), "\n")
## Monthly observations: 212

cat("Date range: ",
    as.character(min(gold_monthly$Date)), "to",
    as.character(max(gold_monthly$Date)), "\n")
## Date range: 2008-01-31 to 2025-08-29

vars_rq2_m <- gold_monthly[, c("Date", "Gold", "USD_Index", "SP500",
                              "Inflation_CPI", "X10Y_Treasury_Yield",
                              "Unemployment")]
colnames(vars_rq2_m)[6] <- "Treasury_10Y"

vars_rq2_m <- vars_rq2_m[complete.cases(vars_rq2_m), ]
cat("Monthly observations after dropping NAs:", nrow(vars_rq2_m), "\n")
## Monthly observations after dropping NAs: 212

vars_numeric_m <- vars_rq2_m[, -1]
var_names_m <- colnames(vars_numeric_m)

adf_results_m <- data.frame(Variable = var_names_m, ADF_Stat = NA,
                             P_Value = NA, Stationary = NA,
                             stringsAsFactors = FALSE)
for (i in seq_along(var_names_m)) {
  test <- adf.test(vars_numeric_m[[var_names_m[i]]],
                   alternative = "stationary")
  adf_results_m$ADF_Stat[i] <- round(test$statistic, 4)
  adf_results_m$P_Value[i] <- round(test$p.value, 4)
}

```

```

    adf_results_m$Stationary[i] <- ifelse(test$p.value < 0.05, "Yes", "No")
  }
  cat("ADF tests on monthly levels:\n")

## ADF tests on monthly levels:

print(adf_results_m)

##      Variable ADF_Stat P_Value Stationary
## 1      Gold      0.9440  0.9900        No
## 2   USD_Index  -3.2439  0.0819        No
## 3     SP500   -1.1572  0.9112        No
## 4 Inflation_CPI -1.1666  0.9097        No
## 5 Treasury_10Y -1.7368  0.6864        No
## 6 Unemployment -3.3964  0.0565        No

adf_diff_m <- data.frame(Variable = var_names_m, ADF_Stat = NA,
                        P_Value = NA, Stationary = NA,
                        stringsAsFactors = FALSE)
for (i in seq_along(var_names_m)) {
  d <- diff(vars_numeric_m[[var_names_m[i]]])
  test <- adf.test(d, alternative = "stationary")
  adf_diff_m$ADF_Stat[i] <- round(test$statistic, 4)
  adf_diff_m$P_Value[i] <- round(test$p.value, 4)
  adf_diff_m$Stationary[i] <- ifelse(test$p.value < 0.05, "Yes", "No")
}
cat("\nADF tests on monthly first differences:\n")

##
## ADF tests on monthly first differences:

print(adf_diff_m)

##      Variable ADF_Stat P_Value Stationary
## 1      Gold  -4.4562   0.01        Yes
## 2   USD_Index -5.4087   0.01        Yes
## 3     SP500  -6.2091   0.01        Yes
## 4 Inflation_CPI -5.2786   0.01        Yes
## 5 Treasury_10Y -6.6038   0.01        Yes
## 6 Unemployment -7.1430   0.01        Yes

y_m <- vars_numeric_m

lag_select_m <- VARselect(y_m, lag.max = 6, type = "const")
cat("Lag selection criteria (monthly):\n")

## Lag selection criteria (monthly):

print(lag_select_m$selection)

## AIC(n)  HQ(n)  SC(n) FPE(n)
##      2      1      1      2

```

```

optimal_lag_m <- as.integer(lag_select_m$selection["AIC(n)"])
cat("Optimal lag at monthly frequency (AIC):", optimal_lag_m, "\n")

## Optimal lag at monthly frequency (AIC): 2

joh_trace_m <- ca.jo(y_m, type = "trace", ecdet = "const",
                     K = optimal_lag_m)
cat("Johansen trace test (monthly):\n")

## Johansen trace test (monthly):

print(summary(joh_trace_m))

##
## #####
## # Johansen-Procedure #
## #####
##
## Test type: trace statistic , without linear trend and constant in
cointegration
##
## Eigenvalues (lambda):
## [1] 2.802780e-01 1.628161e-01 7.634355e-02 5.074945e-02 3.456682e-02
## [6] 1.814997e-02 7.517767e-17
##
## Values of teststatistic and critical values of test:
##
##          test 10pct   5pct   1pct
## r <= 5 |   3.85  7.52   9.24 12.97
## r <= 4 |  11.23 17.85  19.96 24.60
## r <= 3 |  22.17 32.00  34.91 41.07
## r <= 2 |  38.85 49.65  53.12 60.16
## r <= 1 |  76.17 71.86  76.07 84.45
## r = 0 | 145.23 97.18 102.14 111.01
##
## Eigenvectors, normalised to first column:
## (These are the cointegration relations)
##
##          Gold.12  USD_Index.12  SP500.12  Inflation_CPI.12
## Gold.12          1.00000000    1.0000000    1.0000000    1.0000000
## USD_Index.12     20.22881345   143.5820249   23.4554726   13.828865
## SP500.12         -1.07494087    0.1182315   -0.1394203   -1.045769
## Inflation_CPI.12  0.08165783   -40.3373512  -21.3625528    31.376467
## Treasury_10Y.12  -41.77105449   583.0868167  325.2360842   -255.200938
## Unemployment.12  -236.85317031   380.1541901 -163.6301666   -72.113404
## constant         -62.77131630 -8930.6272453 2233.5539842   -6737.085955
##
##          Treasury_10Y.12  Unemployment.12  constant
## Gold.12                  1.00000000    1.0000000    1.0000000
## USD_Index.12              69.3948141    -27.5774249   -1.0103477
## SP500.12                  0.4708225     -0.1671514    0.4428334
## Inflation_CPI.12         -60.9794945    -6.6334769   -30.3133173

```

```

## Treasury_10Y.l2      27.1521074      127.7795828   -30.0827437
## Unemployment.l2      -51.2107725      46.9913455   -70.2308885
## constant             6622.5019741     2610.1928330  5437.0908738
##
## Weights W:
## (This is the loading matrix)
##
##              Gold.l2  USD_Index.l2      SP500.l2  Inflation_CPI.l2
## Gold.d          -2.887550e-02  2.140724e-02  0.0197382171   -1.222959e-02
## USD_Index.d      2.742490e-04 -7.263131e-04 -0.0000799020   -4.288877e-04
## SP500.d          -3.281653e-02  1.277111e-02 -0.0028360199    5.954367e-02
## Inflation_CPI.d -2.450810e-04 -1.053691e-04 -0.0002893598    2.516924e-05
## Treasury_10Y.d   1.941369e-05  2.127511e-05 -0.0000851079    1.017925e-05
## Unemployment.d   -5.052371e-05 -1.670586e-04  0.0004750469    1.168472e-04
##
##              Treasury_10Y.l2  Unemployment.l2      constant
## Gold.d          -1.137835e-03  -3.235714e-03 -5.397748e-16
## USD_Index.d      -4.418035e-04   8.477274e-05 -1.612816e-17
## SP500.d          -1.179395e-02   9.478086e-03 -1.490927e-15
## Inflation_CPI.d   8.256972e-05  -4.155557e-05 -8.005715e-18
## Treasury_10Y.d   -4.496646e-05  -4.167706e-05 -6.882320e-18
## Unemployment.d    1.148941e-05  -4.403524e-05  2.579773e-17

joh_eigen_m <- ca.jo(y_m, type = "eigen", ecdet = "const",
                     K = optimal_lag_m)
cat("\nJohansen maximum eigenvalue test (monthly):\n")

##
## Johansen maximum eigenvalue test (monthly):

print(summary(joh_eigen_m))

##
## #####
## # Johansen-Procedure #
## #####
##
## Test type: maximal eigenvalue statistic (lambda max) , without linear
trend and constant in cointegration
##
## Eigenvalues (lambda):
## [1] 2.802780e-01 1.628161e-01 7.634355e-02 5.074945e-02 3.456682e-02
## [6] 1.814997e-02 7.517767e-17
##
## Values of teststatistic and critical values of test:
##
##              test 10pct  5pct  1pct
## r <= 5 |  3.85  7.52  9.24 12.97
## r <= 4 |  7.39 13.75 15.67 20.20
## r <= 3 | 10.94 19.77 22.00 26.81
## r <= 2 | 16.68 25.56 28.14 33.24
## r <= 1 | 37.32 31.66 34.40 39.79

```

```

## r = 0 | 69.07 37.45 40.30 46.82
##
## Eigenvectors, normalised to first column:
## (These are the cointegration relations)
##
##           Gold.l2  USD_Index.l2  SP500.l2  Inflation_CPI.l2
## Gold.l2      1.0000000      1.0000000      1.0000000      1.0000000
## USD_Index.l2  20.22881345    143.5820249    23.4554726     13.828865
## SP500.l2     -1.07494087      0.1182315     -0.1394203     -1.045769
## Inflation_CPI.l2  0.08165783   -40.3373512   -21.3625528     31.376467
## Treasury_10Y.l2 -41.77105449    583.0868167   325.2360842    -255.200938
## Unemployment.l2 -236.85317031    380.1541901  -163.6301666    -72.113404
## constant     -62.77131630  -8930.6272453  2233.5539842   -6737.085955
##           Treasury_10Y.l2  Unemployment.l2  constant
## Gold.l2      1.0000000      1.0000000      1.0000000
## USD_Index.l2  69.3948141     -27.5774249   -1.0103477
## SP500.l2      0.4708225      -0.1671514     0.4428334
## Inflation_CPI.l2 -60.9794945   -6.6334769   -30.3133173
## Treasury_10Y.l2  27.1521074    127.7795828   -30.0827437
## Unemployment.l2 -51.2107725     46.9913455   -70.2308885
## constant      6622.5019741    2610.1928330  5437.0908738
##
## Weights W:
## (This is the loading matrix)
##
##           Gold.l2  USD_Index.l2  SP500.l2  Inflation_CPI.l2
## Gold.d      -2.887550e-02  2.140724e-02  0.0197382171  -1.222959e-02
## USD_Index.d  2.742490e-04  -7.263131e-04  -0.0000799020  -4.288877e-04
## SP500.d     -3.281653e-02  1.277111e-02  -0.0028360199  5.954367e-02
## Inflation_CPI.d -2.450810e-04  -1.053691e-04  -0.0002893598  2.516924e-05
## Treasury_10Y.d  1.941369e-05  2.127511e-05  -0.0000851079  1.017925e-05
## Unemployment.d -5.052371e-05  -1.670586e-04  0.0004750469  1.168472e-04
##           Treasury_10Y.l2  Unemployment.l2  constant
## Gold.d      -1.137835e-03  -3.235714e-03  -5.397748e-16
## USD_Index.d  -4.418035e-04  8.477274e-05  -1.612816e-17
## SP500.d     -1.179395e-02  9.478086e-03  -1.490927e-15
## Inflation_CPI.d  8.256972e-05  -4.155557e-05  -8.005715e-18
## Treasury_10Y.d  -4.496646e-05  -4.167706e-05  -6.882320e-18
## Unemployment.d  1.148941e-05  -4.403524e-05  2.579773e-17

trace_stats_m <- joh_trace_m@teststat
trace_crit_m  <- joh_trace_m@cval[, 2] # 5% critical values
n_coint_m     <- sum(trace_stats_m > trace_crit_m)

cat("Cointegrating relationships at monthly frequency (5%):",
    n_coint_m, "\n")

## Cointegrating relationships at monthly frequency (5%): 2

```

```

cat("Model:", ifelse(n_coint_m > 0,
                    paste0("VECM (r = ", n_coint_m, ")",
                          "VAR in differences"), "\n\n")

## Model: VECM (r = 2)

if (n_coint_m > 0) {
  vecm_m <- cajorls(joh_trace_m, r = n_coint_m)

  cat("Beta (cointegrating vector, monthly):\n")
  print(round(vecm_m$beta, 4))

  cat("\nAlpha (adjustment coefficients, monthly):\n")
  print(round(vecm_m$rlm$coefficients[1:n_coint_m, , drop = FALSE], 6))

  cat("\nGold equation (monthly):\n")
  print(summary(vecm_m$rlm)$"Response Gold.d")

  vecm_var_m <- vec2var(joh_trace_m, r = n_coint_m)
} else {
  y_diff_m <- as.data.frame(apply(y_m, 2, diff))
  vecm_var_m <- VAR(y_diff_m, p = optimal_lag_m, type = "const")
  cat("Gold equation from VAR in differences (monthly):\n")
  print(summary(vecm_var_m)$varresult$Gold)
}

## Beta (cointegrating vector, monthly):
##          ect1      ect2
## Gold.l2      1.0000    0.0000
## USD_Index.l2    0.0000    1.0000
## SP500.l2     -1.2706    0.0097
## Inflation_CPI.l2  6.7100  -0.3277
## Treasury_10Y.l2 -144.2421   5.0656
## Unemployment.l2 -338.0368   5.0020
## constant     1391.4770 -71.8899
##
## Alpha (adjustment coefficients, monthly):
##          Gold.d USD_Index.d   SP500.d Inflation_CPI.d Treasury_10Y.d
## ect1 -0.007468  -0.000452 -0.020045    -0.000350     0.000041
## ect2  2.489578  -0.098738  1.169863    -0.020087     0.003447
##          Unemployment.d
## ect1      -0.000218
## ect2      -0.025009
##
## Gold equation (monthly):
##
## Call:
## lm(formula = Gold.d ~ ect1 + ect2 + Gold.dl1 + USD_Index.dl1 +
##     SP500.dl1 + Inflation_CPI.dl1 + Treasury_10Y.dl1 + Unemployment.dl1 -
##     1, data = data.mat)

```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -168.285  -42.093   -1.374   40.934  216.202
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## ect1          -0.007468   0.007957  -0.939  0.34906
## ect2           2.489578   0.862362   2.887  0.00431 **
## Gold.dl1       -0.232029   0.078309  -2.963  0.00341 **
## USD_Index.dl1  -4.247626   2.896653  -1.466  0.14410
## SP500.dl1      -0.074818   0.039560  -1.891  0.06002 .
## Inflation_CPI.dl1 -9.136142   7.327388  -1.247  0.21390
## Treasury_10Y.dl1 14.357107  19.395510   0.740  0.46002
## Unemployment.dl1 14.993880   6.646264   2.256  0.02514 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 68.02 on 202 degrees of freedom
## Multiple R-squared:  0.1852, Adjusted R-squared:  0.153
## F-statistic: 5.741 on 8 and 202 DF,  p-value: 1.363e-06

# v7: the trace statistic at r <= 1 clears its 5% critical value by
# only ~0.1 (76.17 vs 76.07), so the rank choice r = 2 is knife-edge.
# Re-derive the headline FEVD under the conservative rank r = 1 to
# show the unemployment ranking does not depend on the rank choice.
if (n_coint_m >= 2) {
  vecm_var_r1 <- vec2var(joh_trace_m, r = 1)
  fevd_r1 <- fevd(vecm_var_r1, n.ahead = 12)$Gold
  cat("FEVD for Gold at 12-month horizon under r = 1:\n")
  print(round(fevd_r1[12, ], 4))
}

## FEVD for Gold at 12-month horizon under r = 1:
##      Gold      USD_Index      SP500 Inflation_CPI Treasury_10Y
##      0.7811      0.0053      0.0052      0.1007      0.0004
## Unemployment
##      0.1074

set.seed(898395)
impulse_vars <- c("USD_Index", "SP500", "Inflation_CPI",
                  "Treasury_10Y", "Unemployment")

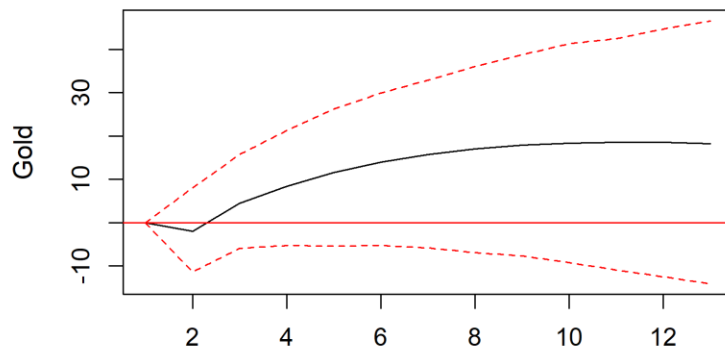
for (impulse_var in impulse_vars) {
  irf_m <- irf(vecm_var_m,
               impulse = impulse_var,
               response = "Gold",
               n.ahead = 12,
               boot     = TRUE,
               runs     = 1000)
}
```

```

par(cex.main = 1.1, cex.axis = 1.05, cex.lab = 1.1)
plot(irf_m,
     main = paste("Monthly: Gold response to", impulse_var, "shock"))
}

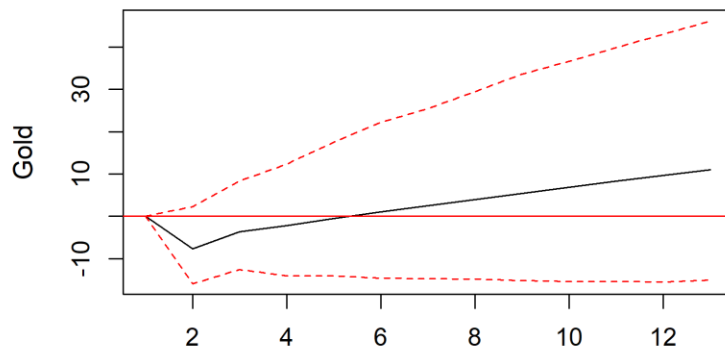
```

Monthly: Gold response to USD_Index shock



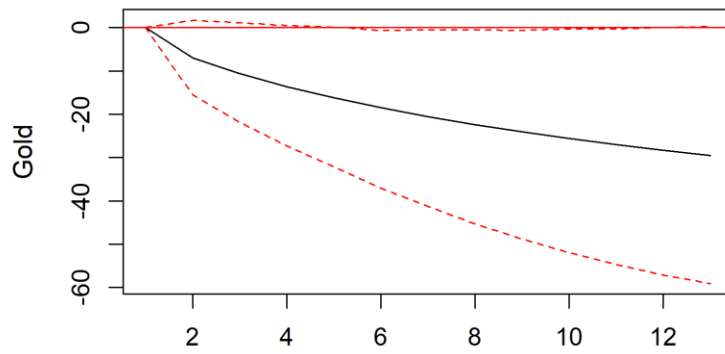
95 % Bootstrap CI, 1000 runs

Monthly: Gold response to SP500 shock



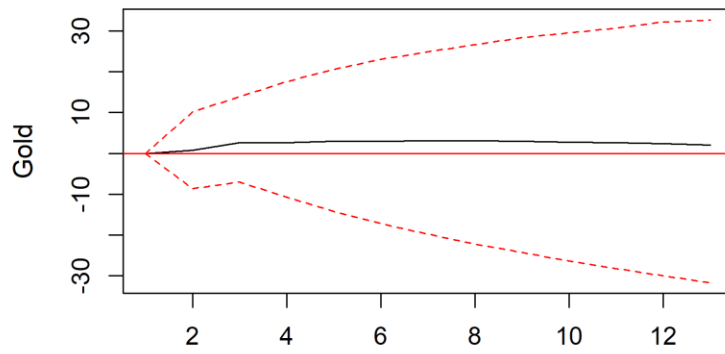
95 % Bootstrap CI, 1000 runs

Monthly: Gold response to Inflation_CPI shock

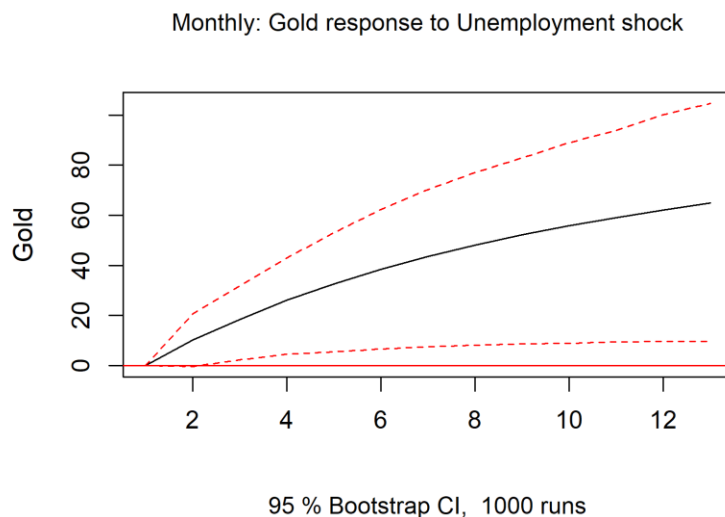


95 % Bootstrap CI, 1000 runs

Monthly: Gold response to Treasury_10Y shock



95 % Bootstrap CI, 1000 runs



```
fevd_m <- fevd(vecm_var_m, n.ahead = 12)

cat("FEVD for Gold (monthly), all horizons:\n")

## FEVD for Gold (monthly), all horizons:
print(round(fevd_m$Gold, 4))

##           Gold USD_Index  SP500 Inflation_CPI Treasury_10Y Unemployment
## [1,] 1.0000    0.0000 0.0000      0.0000      0.0000      0.0000
## [2,] 0.9722    0.0005 0.0074      0.0062      0.0001      0.0136
## [3,] 0.9378    0.0021 0.0063      0.0142      0.0007      0.0390
## [4,] 0.8913    0.0061 0.0050      0.0228      0.0009      0.0738
## [5,] 0.8401    0.0116 0.0039      0.0312      0.0012      0.1120
## [6,] 0.7883    0.0174 0.0032      0.0392      0.0013      0.1507
## [7,] 0.7387    0.0227 0.0028      0.0465      0.0014      0.1880
## [8,] 0.6925    0.0272 0.0028      0.0531      0.0014      0.2229
## [9,] 0.6504    0.0308 0.0031      0.0591      0.0014      0.2551
## [10,] 0.6123    0.0336 0.0037      0.0646      0.0014      0.2845
## [11,] 0.5779    0.0355 0.0045      0.0695      0.0013      0.3113
## [12,] 0.5469    0.0367 0.0054      0.0740      0.0013      0.3357

cat("\nFEVD for Gold (monthly) at 12-month horizon (key row):\n")

##
## FEVD for Gold (monthly) at 12-month horizon (key row):
print(round(fevd_m$Gold[12, ], 4))

##           Gold      USD_Index      SP500 Inflation_CPI Treasury_10Y
##           0.5469      0.0367      0.0054      0.0740      0.0013
```

```

## Unemployment
##      0.3357

# v7: the FEVD is Cholesky-identified, so the shares depend on the
# variable ordering (baseline: Gold first). The headline result --
# unemployment is the largest macro contributor to gold's 12-month
# FEVD -- must not hinge on that choice. Re-estimate under several
# plausible orderings and report unemployment's 12-month share.
fevd_unemp_share_m <- function(order_vec) {
  joh_o <- ca.jo(y_m[, order_vec], type = "trace", ecdet = "const",
                 K = optimal_lag_m)
  vv_o <- vec2var(joh_o, r = n_coint_m)
  fg <- fevd(vv_o, n.ahead = 12)$Gold
  fg[12, "Unemployment"]
}

orderings_m <- list(
  baseline = c("Gold", "USD_Index", "SP500", "Inflation_CPI",
               "Treasury_10Y", "Unemployment"),
  macro_first = c("Unemployment", "Inflation_CPI", "Treasury_10Y",
                  "USD_Index", "SP500", "Gold"),
  gold_last = c("USD_Index", "SP500", "Inflation_CPI",
                "Treasury_10Y", "Unemployment", "Gold"),
  unemp_first = c("Unemployment", "Gold", "USD_Index", "SP500",
                  "Inflation_CPI", "Treasury_10Y")
)

robust_m <- data.frame(
  Ordering = names(orderings_m),
  Unemp_share_12m = sapply(orderings_m, function(o)
    round(as.numeric(fevd_unemp_share_m(o)), 4)),
  stringsAsFactors = FALSE
)
rownames(robust_m) <- NULL
cat("Unemployment's share of Gold's 12-month FEVD by ordering:\n")

## Unemployment's share of Gold's 12-month FEVD by ordering:

print(robust_m)

##      Ordering Unemp_share_12m
## 1 baseline      0.3357
## 2 macro_first    0.4498
## 3 gold_last      0.4299
## 4 unemp_first    0.4498

# =====
# RQ2 ROBUSTNESS: is the unemployment FEVD share a crisis artifact?
# =====
# The headline result -- unemployment ~ a third of gold's 12-month
# FEVD at r = 2 -- is re-estimated on subsamples that exclude the

```

```

# GFC (2008-09) and/or the COVID shock (2020), plus a
# leave-one-year-out jackknife. The cointegrating rank is held fixed
# ( $r = 2$ , and  $r = 1$ ) and the lag length is held at optimal_lag_m, so
# this isolates CRISIS INFLUENCE from the rank-selection question.
# fevd_gold_12m() reuses the same ca.jo -> vec2var -> vars::fevd path
# as the main analysis, so the baseline row reproduces Table 6.5.
#
# NOTE on the "Drop ..." rows: deleting calendar years from the
# middle of the series splices the remaining months together, which
# contaminates one differenced observation at each join. The contiguous
# SUBSAMPLE rows (Pre-COVID 2008-2019, Post-GFC 2010-2025,
# No-crisis core 2010-2019) have no internal splice and tell the same
# story, so the conclusion does not rest on the splice.

fevd_gold_12m <- function(dat, rank) {
  y <- dat[, c("Gold", "USD_Index", "SP500", "Inflation_CPI",
               "Treasury_10Y", "Unemployment")]
  joh <- ca.jo(y, type = "trace", ecdet = "const", K = optimal_lag_m)
  fg <- vars::fevd(vec2var(joh, r = rank), n.ahead = 12)$Gold
  round(fg[12, ] * 100, 1)
}

vars_rq2_m$yr <- as.integer(format(vars_rq2_m$Date, "%Y"))

scen <- list(
  "Full 2008-2025 (baseline)" = vars_rq2_m,
  "Drop 2008-09 (GFC)"       = subset(vars_rq2_m, !(yr %in% c(2008, 2009))),
  "Drop 2020 (COVID)"       = subset(vars_rq2_m, yr != 2020),
  "Drop GFC + COVID"        = subset(vars_rq2_m, !(yr %in% c(2008, 2009,
2020))),
  "Pre-COVID 2008-2019"     = subset(vars_rq2_m, yr <= 2019),
  "Post-GFC 2010-2025"     = subset(vars_rq2_m, yr >= 2010),
  "No-crisis core 2010-2019" = subset(vars_rq2_m, yr >= 2010 & yr <= 2019)
)

rob <- do.call(rbind, lapply(names(scen), function(nm) {
  f2 <- fevd_gold_12m(scen[[nm]], rank = 2)
  f1 <- fevd_gold_12m(scen[[nm]], rank = 1)
  data.frame(
    Scenario      = nm,
    n             = nrow(scen[[nm]]),
    Unemp_r2      = as.numeric(f2["Unemployment"]),
    CPI_r2        = as.numeric(f2["Inflation_CPI"]),
    Top_macro_r2  = names(which.max(f2[setdiff(names(f2), "Gold")])),
    Unemp_r1      = as.numeric(f1["Unemployment"]),
    stringsAsFactors = FALSE
  )
}))
rownames(rob) <- NULL

```

```

cat("Unemployment's 12-month FEVD share (%) across crisis-exclusion
samples:\n")

## Unemployment's 12-month FEVD share (%) across crisis-exclusion samples:

cat("(rank held at r=2 and r=1; lag held at optimal_lag_m)\n\n")

## (rank held at r=2 and r=1; lag held at optimal_lag_m)

print(rob)

##
##          Scenario    n Unemp_r2 CPI_r2 Top_macro_r2 Unemp_r1
## 1 Full 2008-2025 (baseline) 212    33.6    7.4 Unemployment    10.7
## 2      Drop 2008-09 (GFC) 188    34.4    8.3 Unemployment    11.4
## 3      Drop 2020 (COVID) 200    10.7    0.6 Unemployment     4.9
## 4      Drop GFC + COVID 176     6.8    0.2 Treasury_10Y     3.8
## 5    Pre-COVID 2008-2019 144     2.1    0.8 Unemployment     2.1
## 6    Post-GFC 2010-2025 188    34.4    8.3 Unemployment    11.4
## 7 No-crisis core 2010-2019 120     1.7    0.2   USD_Index     0.8

# Leave-one-year-out jackknife (r = 2): which single year drives the share?
yrs <- sort(unique(vars_rq2_m$yr))
jk  <- sapply(yrs, function(Y)
  as.numeric(fevd_gold_12m(subset(vars_rq2_m, yr != Y), rank =
2)["Unemployment"])))
names(jk) <- yrs
cat("\nLeave-one-year-out jackknife (r=2): unemployment 12m FEVD share with
each year removed.\n")

##
## Leave-one-year-out jackknife (r=2): unemployment 12m FEVD share with each
year removed.

cat("Lowest value = most influential year.\n")

## Lowest value = most influential year.

print(sort(jk))

## 2020 2025 2024 2013 2018 2017 2019 2015 2016 2014 2022 2008 2011 2012 2010
2021
## 10.7 24.3 28.7 32.1 32.4 33.2 33.4 34.0 34.2 34.3 34.5 34.9 35.7 35.8 39.5
41.1
## 2009 2023
## 41.7 42.5

if (requireNamespace("knitr", quietly = TRUE)) {
  kable(rob, align = "lcccc",
    caption = "RQ2 robustness: unemployment's share of gold's 12-month
FEVD when crisis years are excluded.")
}

```

RQ2 robustness: unemployment's share of gold's 12-month FEVD when crisis years are excluded.

Scenario	n	Unemp_r2	CPI_r2	Top_macro_r2	Unemp_r1
Full 2008-2025 (baseline)	212	33.6	7.4	Unemployment	10.7
Drop 2008-09 (GFC)	188	34.4	8.3	Unemployment	11.4
Drop 2020 (COVID)	200	10.7	0.6	Unemployment	4.9
Drop GFC + COVID	176	6.8	0.2	Treasury_10Y	3.8
Pre-COVID 2008-2019	144	2.1	0.8	Unemployment	2.1
Post-GFC 2010-2025	188	34.4	8.3	Unemployment	11.4
No-crisis core 2010-2019	120	1.7	0.2	USD_Index	0.8

*# Monthly FEVD plotted with the same clean, percent-stacked style as
the daily version, so the two figures are directly comparable.*

```
fevd_m_gold <- fevd_m$Gold
fevd_m_df <- as.data.frame(fevd_m_gold)
fevd_m_df$Horizon <- seq_len(nrow(fevd_m_df))
fevd_m_melt <- melt(fevd_m_df, id.vars = "Horizon",
                    variable.name = "Variable", value.name = "Proportion")

ord_m <- names(sort(fevd_m_gold[nrow(fevd_m_gold), ], decreasing = FALSE))
clean_labels <- c(
  Gold = "Gold (own)",
  USD_Index = "USD Index",
  SP500 = "S&P 500",
  Inflation_CPI = "Inflation (CPI)",
  Treasury_10Y = "10Y Treasury",
  Unemployment = "Unemployment"
)
fevd_m_melt$Variable <- factor(fevd_m_melt$Variable, levels = ord_m,
                              labels = clean_labels[ord_m])

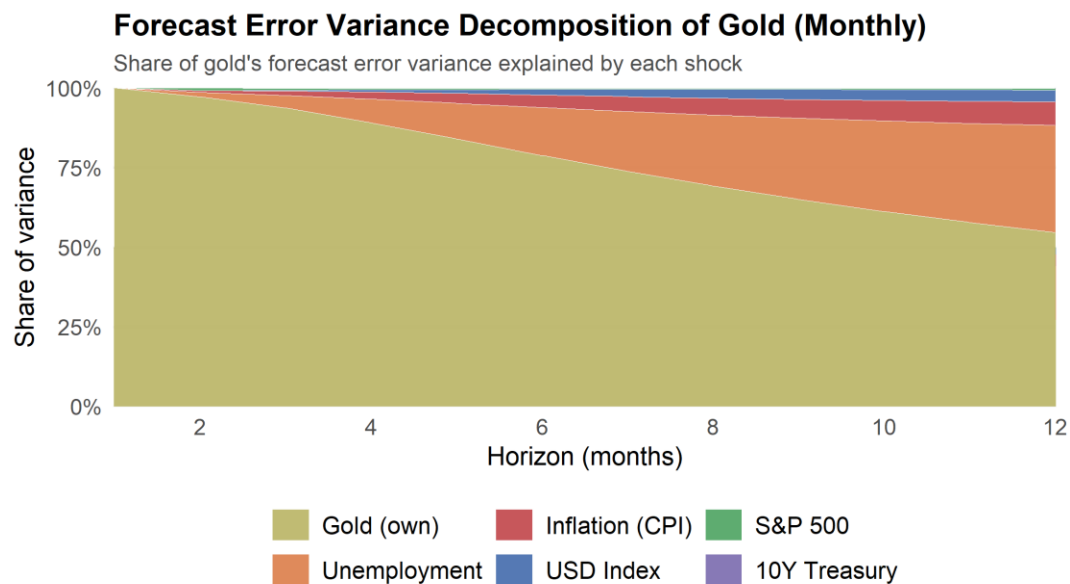
fevd_cols <- c(
  "Gold (own)" = "#BDB76B",
  "USD Index" = "#4C72B0",
  "S&P 500" = "#55A868",
  "Inflation (CPI)" = "#C44E52",
  "10Y Treasury" = "#8172B2",
  "Unemployment" = "#DD8452"
)

ggplot(fevd_m_melt, aes(x = Horizon, y = Proportion, fill = Variable)) +
  geom_area(position = "fill", colour = "white", linewidth = 0.15, alpha =
0.95) +
  scale_fill_manual(values = fevd_cols, breaks =
rev(levels(fevd_m_melt$Variable))) +
```

```

scale_y_continuous(labels = scales::percent_format(accuracy = 1),
  expand = c(0, 0)) +
scale_x_continuous(breaks = seq(2, 12, 2), expand = c(0, 0)) +
labs(title = "Forecast Error Variance Decomposition of Gold (Monthly)",
  subtitle = "Share of gold's forecast error variance explained by each
shock",
  x = "Horizon (months)", y = "Share of variance", fill = NULL) +
theme_thesis() +
theme(legend.position = "bottom",
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  plot.title = element_text(face = "bold"))

```



```

serial_test_m <- serial.test(vecm_var_m, lags.pt = 12,
  type = "PT.asymptotic")
cat("Portmanteau test for serial correlation (monthly):\n")

## Portmanteau test for serial correlation (monthly):

print(serial_test_m)

##
## Portmanteau Test (asymptotic)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 435.45, df = 366, p-value = 0.007245

## $serial
##
## Portmanteau Test (asymptotic)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 435.45, df = 366, p-value = 0.007245

```

```

norm_test_m <- normality.test(vecm_var_m, multivariate.only = TRUE)
cat("\nMultivariate normality test (monthly):\n")

##
## Multivariate normality test (monthly):

print(norm_test_m)

## $JB
##
## JB-Test (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 74814, df = 12, p-value < 2.2e-16
##
## $Skewness
##
## Skewness only (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 2158.9, df = 6, p-value < 2.2e-16
##
## $Kurtosis
##
## Kurtosis only (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 72655, df = 6, p-value < 2.2e-16

## $jb.mul
## $jb.mul$JB
##
## JB-Test (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 74814, df = 12, p-value < 2.2e-16
##
## $jb.mul$Skewness
##
## Skewness only (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 2158.9, df = 6, p-value < 2.2e-16
##
## $jb.mul$Kurtosis
##
## Kurtosis only (multivariate)

```

```

##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 72655, df = 6, p-value < 2.2e-16

arch_test_m <- arch.test(vecm_var_m, lags.multi = 3)
cat("\nMultivariate ARCH test (monthly):\n")

##
## Multivariate ARCH test (monthly):

print(arch_test_m)

##
## ARCH (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 2052.9, df = 1323, p-value < 2.2e-16

## $arch.mul
##
## ARCH (multivariate)
##
## data: Residuals of VAR object vecm_var_m
## Chi-squared = 2052.9, df = 1323, p-value < 2.2e-16

# Toda-Yamamoto Granger causality at monthly frequency, mirroring
# the daily block. Same logic: augmented VAR in levels with
# (p + d_max) lags, Wald-test only the first p lags.
dmax_m <- 1L
p_ty_m <- optimal_lag_m
K_ty_m <- p_ty_m + dmax_m

var_ty_m <- VAR(y_m, p = K_ty_m, type = "both")
predictors_m <- var_names_m[var_names_m != "Gold"]
gold_eq_m <- var_ty_m$varresult[["Gold"]]
coef_gold_m <- coef(gold_eq_m)
V_gold_m <- vcov(gold_eq_m)

granger_ty_m <- data.frame(Variable = predictors_m, Chisq = NA,
                           df = NA, P_Value = NA, Significant = NA,
                           stringsAsFactors = FALSE)
for (i in seq_along(predictors_m)) {
  v <- predictors_m[i]
  target_terms <- paste0(v, ".1", seq_len(p_ty_m))
  idx <- which(names(coef_gold_m) %in% target_terms)
  wt <- aod::wald.test(b = coef_gold_m, Sigma = V_gold_m, Terms = idx)
  granger_ty_m$Chisq[i] <- round(as.numeric(wt$result$chi2["chi2"]), 4)
  granger_ty_m$df[i] <- as.numeric(wt$result$chi2["df"])
  granger_ty_m$P_Value[i] <- round(as.numeric(wt$result$chi2["P"]), 4)
  granger_ty_m$Significant[i] <- ifelse(as.numeric(wt$result$chi2["P"]) <
0.05, "Yes", "No")

```

```

}
cat("Toda-Yamamoto Granger causality toward Gold (monthly):\n")

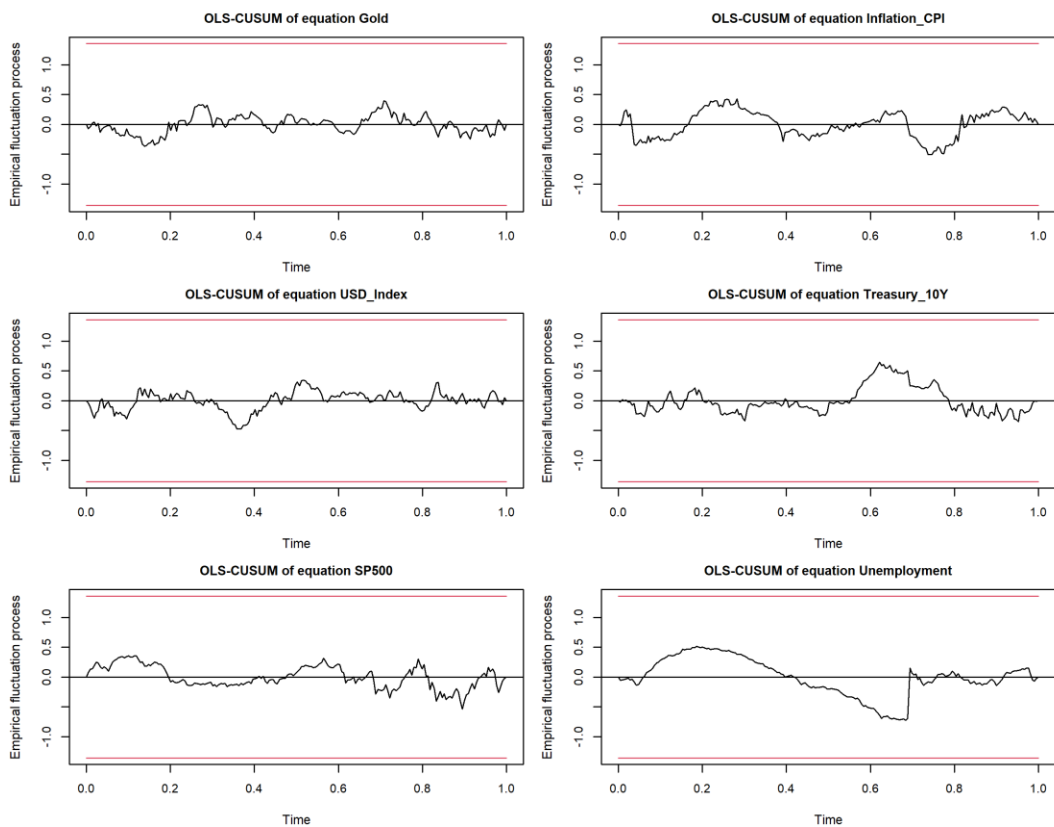
## Toda-Yamamoto Granger causality toward Gold (monthly):

print(granger_ty_m)

##      Variable  Chisq df P_Value Significant
## 1    USD_Index 2.3232  2  0.3130          No
## 2      SP500 4.4868  2  0.1061          No
## 3 Inflation_CPI 0.7816  2  0.6765          No
## 4 Treasury_10Y 0.1748  2  0.9163          No
## 5 Unemployment 5.5689  2  0.0618          No

# v7: OLS-CUSUM stability for the MONTHLY system. The previous
# version of thesis Figure 6.10 was generated from the daily levels
# VAR; the thesis reports the monthly VECM, so the stability
# diagnostic must come from the monthly system. Reuses var_ty_m from
# the Toda-Yamamoto chunk above.
op <- par(cex.main = 1.0, cex.axis = 0.95, cex.lab = 1.0,
          mar = c(4, 4, 2.5, 1))
plot(stability(var_ty_m), nc = 2)

```



```
par(op)
```

v9: the lag-augmented LEVELS VAR (Toda-Yamamoto) is expected to

```

# carry a root on/above the unit circle when the data are I(1) and
# cointegrated -- that near-unit root IS the cointegration, not a
# misspecification. Parameter stability is judged by the OLS-CUSUM
# paths above, not by this root. Labelled accordingly to avoid a
# misleading "UNSTABLE" flag in the knitted output.
roots_m_chk <- roots(var_ty_m)
cat("Max characteristic root of lag-augmented levels VAR (monthly):",
    round(max(roots_m_chk), 4),
    "-- a near-unit root is expected here (I(1)/cointegrated data);\n",
    "    parameter stability is assessed by the OLS-CUSUM paths above.\n")

## Max characteristic root of lag-augmented levels VAR (monthly): 1.0154 -- a
## near-unit root is expected here (I(1)/cointegrated data);
##     parameter stability is assessed by the OLS-CUSUM paths above.

cat("=====\n")

## =====

cat("MONTHLY VECM SUMMARY\n")

## MONTHLY VECM SUMMARY

cat("=====\n")

## =====

cat("Sample period:           ",
    as.character(min(vars_rq2_m$Date)), "to",
    as.character(max(vars_rq2_m$Date)), "\n")

## Sample period:                2008-01-31 to 2025-08-29

cat("Monthly observations:      ", nrow(vars_rq2_m), "\n")

## Monthly observations:         212

cat("Optimal lag (AIC):         ", optimal_lag_m, "\n")

## Optimal lag (AIC):            2

cat("Cointegrating relations (5%):", n_coint_m, "\n")

## Cointegrating relations (5%): 2

cat("Model:                     ",
    ifelse(n_coint_m > 0,
        paste0("VECM (r = ", n_coint_m, ")"),
        "VAR in differences"), "\n")

## Model:                        VECM (r = 2)

cat("-----\n")

```

```
## -----

cat("FEVD for Gold at 12-month horizon:\n")

## FEVD for Gold at 12-month horizon:

print(round(fevd_m$Gold[12, ], 4))

##          Gold      USD_Index      SP500 Inflation_CPI  Treasury_10Y
##          0.5469      0.0367      0.0054      0.0740      0.0013
## Unemployment
##          0.3357

cat("===== \n")

## =====

# RQ3: How do the volatility and correlation patterns of Bitcoin
# stack up against Gold and the S&P 500?

# NOTE (fix #1 & #2): reuse the already-loaded gold_data instead of
# re-reading and overwriting the CSV. We keep only rows where
# Bitcoin exists, then compute returns. The official VIX_CBOE
# column travels with gold_data and is used for the regime split.

df_full <- gold_data[!is.na(gold_data$Bitcoin), ]

# Daily returns
df_full$Gold_Return <- c(NA, diff(df_full$Gold) / df_full$Gold[-
nrow(df_full)])
df_full$SP500_Return <- c(NA, diff(df_full$SP500) / df_full$SP500[-
nrow(df_full)])
df_full$BTC_Return <- c(NA, diff(df_full$Bitcoin) / df_full$Bitcoin[-
nrow(df_full)])

# Remove first row (NA from return calculation)
df_full <- df_full[-1, ]

gold_vol <- sd(df_full$Gold_Return, na.rm = TRUE) * sqrt(252) * 100
sp_vol <- sd(df_full$SP500_Return, na.rm = TRUE) * sqrt(252) * 100
btc_vol <- sd(df_full$BTC_Return, na.rm = TRUE) * sqrt(252) * 100

vol_table <- data.frame(
  Asset = c("Gold", "S&P 500", "Bitcoin"),
  `Annualized Volatility (%)` = round(c(gold_vol, sp_vol, btc_vol), 2),
  `Relative to Gold` = round(c(1, sp_vol/gold_vol, btc_vol/gold_vol), 1),
  check.names = FALSE
)

kable(vol_table, align = "lcc")
```

Asset	Annualized Volatility (%)	Relative to Gold
Gold	15.07	1.0
S&P 500	17.98	1.2
Bitcoin	67.44	4.5

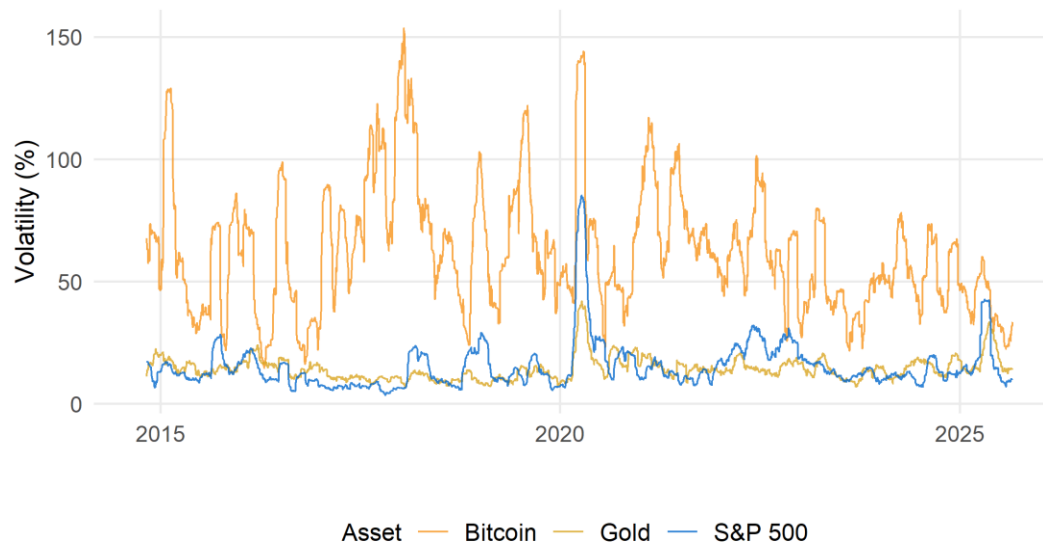
```
# 30-day rolling volatility
df_full$Gold_Vol_30d <- rollapply(df_full$Gold_Return, 30, sd, fill = NA,
align = "right") * sqrt(252) * 100
df_full$SP500_Vol_30d <- rollapply(df_full$SP500_Return, 30, sd, fill = NA,
align = "right") * sqrt(252) * 100
df_full$BTC_Vol_30d <- rollapply(df_full$BTC_Return, 30, sd, fill = NA,
align = "right") * sqrt(252) * 100
vol_plot_data <- data.frame(
  Date = rep(df_full$Date, 3),
  Volatility = c(df_full$Gold_Vol_30d, df_full$SP500_Vol_30d,
df_full$BTC_Vol_30d),
  Asset = factor(rep(c("Gold", "S&P 500", "Bitcoin"), each = nrow(df_full)),
    levels = c("Bitcoin", "Gold", "S&P 500"))
)

ggplot(vol_plot_data, aes(x = Date, y = Volatility, color = Asset)) +
  geom_line(linewidth = 0.5, alpha = 0.8) +
  scale_color_manual(values = c("Bitcoin" = "#F7931A", "Gold" = "#DAA520",
"S&P 500" = "#0066CC")) +
  labs(title = "30-Day Rolling Volatility (Annualized)",
    subtitle = "Bitcoin vs gold vs S&P 500, based on daily returns",
    y = "Volatility (%)", x = "") +
  theme_thesis() +
  theme(legend.position = "bottom",
    plot.title = element_text(face = "bold"))

## Warning: Removed 87 rows containing missing values or values outside the
scale range
## (`geom_line()`).
```

30-Day Rolling Volatility (Annualized)

Bitcoin vs gold vs S&P 500, based on daily returns



```
vol_summary <- data.frame(  
  Asset = c("Gold", "S&P 500", "Bitcoin"),  
  Mean = round(c(mean(df_full$Gold_Vol_30d, na.rm = TRUE),  
                  mean(df_full$SP500_Vol_30d, na.rm = TRUE),  
                  mean(df_full$BTC_Vol_30d, na.rm = TRUE)), 2),  
  Median = round(c(median(df_full$Gold_Vol_30d, na.rm = TRUE),  
                   median(df_full$SP500_Vol_30d, na.rm = TRUE),  
                   median(df_full$BTC_Vol_30d, na.rm = TRUE)), 2),  
  Min = round(c(min(df_full$Gold_Vol_30d, na.rm = TRUE),  
                min(df_full$SP500_Vol_30d, na.rm = TRUE),  
                min(df_full$BTC_Vol_30d, na.rm = TRUE)), 2),  
  Max = round(c(max(df_full$Gold_Vol_30d, na.rm = TRUE),  
                max(df_full$SP500_Vol_30d, na.rm = TRUE),  
                max(df_full$BTC_Vol_30d, na.rm = TRUE)), 2)  
)  
  
kable(vol_summary, align = "lccc")
```

Asset	Mean	Median	Min	Max
Gold	14.39	13.78	6.86	42.05
S&P 500	15.26	12.59	3.57	85.31
Bitcoin	62.17	58.76	14.57	153.52

Symmetric GARCH(1,1) specification (baseline)

```
garch_spec <- ugarchspec(  
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),  
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE),  
  distribution.model = "std" # Student-t for fat tails  
)
```

```

# Fit GARCH to Gold
gold_garch <- ugarchfit(spec = garch_spec, data = df_full$Gold_Return)
cat("=== GOLD GARCH(1,1) ===\n")

## === GOLD GARCH(1,1) ===

print(gold_garch)

##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(0,0,0)
## Distribution   : std
##
## Optimal Parameters
## -----
##      Estimate  Std. Error   t value Pr(>|t|)
## mu      0.000454   0.000149   3.05156 0.002277
## omega    0.000001   0.000001   0.71656 0.473647
## alpha1   0.034210   0.008939   3.82689 0.000130
## beta1    0.956600   0.007030 136.07284 0.000000
## shape    4.442618   0.852911   5.20877 0.000000
##
## Robust Standard Errors:
##      Estimate  Std. Error   t value Pr(>|t|)
## mu      0.000454   0.000181   2.509822 0.012079
## omega    0.000001   0.000015   0.069792 0.944359
## alpha1   0.034210   0.084576   0.404485 0.685856
## beta1    0.956600   0.066340 14.419723 0.000000
## shape    4.442618   7.765839   0.572072 0.567273
##
## LogLikelihood : 9140.143
##
## Information Criteria
## -----
##
## Akaike          -6.6365
## Bayes           -6.6257
## Shibata         -6.6365
## Hannan-Quinn    -6.6326
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##                                statistic p-value
## Lag[1]                                1.204 0.2724

```

```

## Lag[2*(p+q)+(p+q)-1][2]      1.219  0.4322
## Lag[4*(p+q)+(p+q)-1][5]      3.869  0.2709
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##                               statistic p-value
## Lag[1]                        1.216  0.2702
## Lag[2*(p+q)+(p+q)-1][5]      3.691  0.2952
## Lag[4*(p+q)+(p+q)-1][9]      4.811  0.4577
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##           Statistic Shape Scale P-Value
## ARCH Lag[3]      1.689 0.500 2.000  0.1938
## ARCH Lag[5]      2.930 1.440 1.667  0.3001
## ARCH Lag[7]      3.140 2.315 1.543  0.4875
##
## Nyblom stability test
## -----
## Joint Statistic:  554.8575
## Individual Statistics:
## mu      0.8429
## omega   69.8801
## alpha1  0.4191
## beta1   0.3000
## shape   0.6745
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.28 1.47 1.88
## Individual Statistic:  0.35 0.47 0.75
##
## Sign Bias Test
## -----
##           t-value  prob sig
## Sign Bias      0.0223 0.9822
## Negative Sign Bias 1.3509 0.1769
## Positive Sign Bias 0.5505 0.5820
## Joint Effect      2.4676 0.4812
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      22.76      0.2482
## 2    30      28.06      0.5145
## 3    40      40.42      0.4075
## 4    50      51.34      0.3821

```

```
##
##
## Elapsed time : 0.3887699

# Fit GARCH to Bitcoin
btc_garch <- ugarchfit(spec = garch_spec, data = df_full$BTC_Return)
cat("=== BITCOIN GARCH(1,1) ===\n")

## === BITCOIN GARCH(1,1) ===

print(btc_garch)

##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(0,0,0)
## Distribution   : std
##
## Optimal Parameters
## -----
##          Estimate  Std. Error  t value Pr(>|t|)
## mu        0.001600    0.000496   3.2230 0.001269
## omega      0.000042    0.000014   2.9451 0.003228
## alpha1     0.115510    0.017818   6.4829 0.000000
## beta1      0.883490    0.017768  49.7241 0.000000
## shape      3.064182    0.167200  18.3265 0.000000
##
## Robust Standard Errors:
##          Estimate  Std. Error  t value Pr(>|t|)
## mu        0.001600    0.000496   3.2232 0.001268
## omega      0.000042    0.000023   1.8305 0.067181
## alpha1     0.115510    0.020972   5.5079 0.000000
## beta1      0.883490    0.025624  34.4793 0.000000
## shape      3.064182    0.162076  18.9058 0.000000
##
## LogLikelihood : 5303.852
##
## Information Criteria
## -----
##
## Akaike          -3.8495
## Bayes           -3.8388
## Shibata         -3.8495
## Hannan-Quinn    -3.8456
##
## Weighted Ljung-Box Test on Standardized Residuals
```

```

## -----
##                               statistic p-value
## Lag[1]                        4.719 0.02982
## Lag[2*(p+q)+(p+q)-1][2]      4.895 0.04338
## Lag[4*(p+q)+(p+q)-1][5]      8.406 0.02334
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##                               statistic p-value
## Lag[1]                        0.03582 0.8499
## Lag[2*(p+q)+(p+q)-1][5]      2.45558 0.5154
## Lag[4*(p+q)+(p+q)-1][9]      3.51135 0.6724
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##                               Statistic Shape Scale P-Value
## ARCH Lag[3]                   2.877 0.500 2.000 0.08986
## ARCH Lag[5]                   3.598 1.440 1.667 0.21402
## ARCH Lag[7]                   3.778 2.315 1.543 0.37970
##
## Nyblom stability test
## -----
## Joint Statistic: 8.1284
## Individual Statistics:
## mu      0.3913
## omega   0.9737
## alpha1  0.3573
## beta1   0.5289
## shape   0.7327
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.28 1.47 1.88
## Individual Statistic: 0.35 0.47 0.75
##
## Sign Bias Test
## -----
##                               t-value   prob sig
## Sign Bias                0.02161 0.9828
## Negative Sign Bias       0.02178 0.9826
## Positive Sign Bias       1.06825 0.2855
## Joint Effect              1.57227 0.6657
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      48.36   0.0002276

```

```

## 2      30      58.53      0.0009369
## 3      40      75.78      0.0003777
## 4      50      90.86      0.0002603
##
##
## Elapsed time : 0.4453321

# Fit GARCH to S&P 500
sp_garch <- ugarchfit(spec = garch_spec, data = df_full$SP500_Return)
cat("=== S&P 500 GARCH(1,1) ===\n")

## === S&P 500 GARCH(1,1) ===

print(sp_garch)

##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(0,0,0)
## Distribution   : std
##
## Optimal Parameters
## -----
##          Estimate  Std. Error  t value Pr(>|t|)
## mu        0.000940    0.000125   7.5001 0.000000
## omega      0.000002    0.000001   1.6910 0.090844
## alpha1     0.181916    0.026522   6.8591 0.000000
## beta1      0.814923    0.022791  35.7564 0.000000
## shape      5.560640    0.604476   9.1991 0.000000
##
## Robust Standard Errors:
##          Estimate  Std. Error  t value Pr(>|t|)
## mu        0.000940    0.000104   9.04503 0.000000
## omega      0.000002    0.000005   0.47399 0.635505
## alpha1     0.181916    0.065002   2.79862 0.005132
## beta1      0.814923    0.063160  12.90246 0.000000
## shape      5.560640    0.852029   6.52635 0.000000
##
## LogLikelihood : 9208.713
##
## Information Criteria
## -----
##
## Akaike          -6.6863
## Bayes           -6.6756
## Shibata         -6.6863

```

```

## Hannan-Quinn -6.6824
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##               statistic p-value
## Lag[1]          1.357  0.2440
## Lag[2*(p+q)+(p+q)-1][2]  1.383  0.3892
## Lag[4*(p+q)+(p+q)-1][5]  2.506  0.5046
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##               statistic p-value
## Lag[1]          0.5869  0.4436
## Lag[2*(p+q)+(p+q)-1][5]  2.9933  0.4083
## Lag[4*(p+q)+(p+q)-1][9]  6.0484  0.2927
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##               Statistic Shape Scale P-Value
## ARCH Lag[3]    0.2061 0.500 2.000  0.6498
## ARCH Lag[5]    4.8570 1.440 1.667  0.1109
## ARCH Lag[7]    6.4815 2.315 1.543  0.1120
##
## Nyblom stability test
## -----
## Joint Statistic:  29.1904
## Individual Statistics:
## mu      0.3938
## omega   11.2378
## alpha1  0.4153
## beta1   0.8756
## shape   0.6143
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.28 1.47 1.88
## Individual Statistic:  0.35 0.47 0.75
##
## Sign Bias Test
## -----
##               t-value      prob sig
## Sign Bias      3.3134 0.0009337 ***
## Negative Sign Bias  0.4335 0.6646882
## Positive Sign Bias  0.5031 0.6149020
## Joint Effect     20.1463 0.0001583 ***
##
##
## Adjusted Pearson Goodness-of-Fit Test:

```

```

## -----
##   group statistic p-value(g-1)
## 1    20      66.78   3.125e-07
## 2    30      94.19   8.142e-09
## 3    40     113.24   3.664e-09
## 4    50     118.07   1.246e-07
##
##
## Elapsed time : 0.40223

# =====
# Asymmetric GARCH robustness (GJR-GARCH)   [smaller fix]
# =====
# A symmetric sGARCH treats positive and negative shocks of the
# same size identically. Defensive-asset arguments hinge on
# behaviour during downturns, so we re-fit each asset with a
# GJR-GARCH(1,1), which allows a "leverage" term gamma1: a
# significant positive gamma1 means negative shocks raise
# volatility more than positive shocks of equal size. We compare
# information criteria and report gamma1.

gjr_spec <- ugarchspec(
  variance.model = list(model = "gjrGARCH", garchOrder = c(1, 1)),
  mean.model     = list(armaOrder = c(0, 0), include.mean = TRUE),
  distribution.model = "std"
)

fit_safe <- function(spec, x) {
  out <- tryCatch(ugarchfit(spec = spec, data = x, solver = "hybrid"),
    error = function(e) NULL)
  out
}

gold_gjr <- fit_safe(gjr_spec, df_full$Gold_Return)
btc_gjr  <- fit_safe(gjr_spec, df_full$BTC_Return)
sp_gjr   <- fit_safe(gjr_spec, df_full$SP500_Return)

get_gamma <- function(fit) if (is.null(fit)) NA else
as.numeric(coef(fit)["gamma1"])
get_aic   <- function(fit) if (is.null(fit)) NA else
infocriteria(fit)["Akaike", 1]

asym_tbl <- data.frame(
  Asset      = c("Gold", "Bitcoin", "S&P 500"),
  sGARCH_AIC = round(c(infocriteria(gold_garch)["Akaike", 1],
    infocriteria(btc_garch)["Akaike", 1],
    infocriteria(sp_garch)["Akaike", 1]), 4),
  gjrGARCH_AIC = round(c(get_aic(gold_gjr), get_aic(btc_gjr),
    get_aic(sp_gjr)), 4),

```

```

    gjr_gamma1      = round(c(get_gamma(gold_gjr), get_gamma(btc_gjr),
get_gamma(sp_gjr)), 4),
    stringsAsFactors = FALSE
)
cat("Asymmetric (GJR) vs symmetric GARCH -- lower AIC is better;\n")

## Asymmetric (GJR) vs symmetric GARCH -- lower AIC is better;

cat("positive gamma1 indicates a leverage/asymmetry effect;\n")

## positive gamma1 indicates a leverage/asymmetry effect:

print(asym_tbl)

##      Asset sGARCH_AIC gjrGARCH_AIC gjr_gamma1
## 1    Gold   -6.6365    -6.6382    -0.0349
## 2 Bitcoin  -3.8495    -3.8488    -0.0053
## 3 S&P 500  -6.6863    -6.7221     0.2987

df_full$Gold_GARCH_Vol  <- as.numeric(sigma(gold_garch)) * sqrt(252) * 100
df_full$BTC_GARCH_Vol   <- as.numeric(sigma(btc_garch)) * sqrt(252) * 100
df_full$SP500_GARCH_Vol <- as.numeric(sigma(sp_garch)) * sqrt(252) * 100

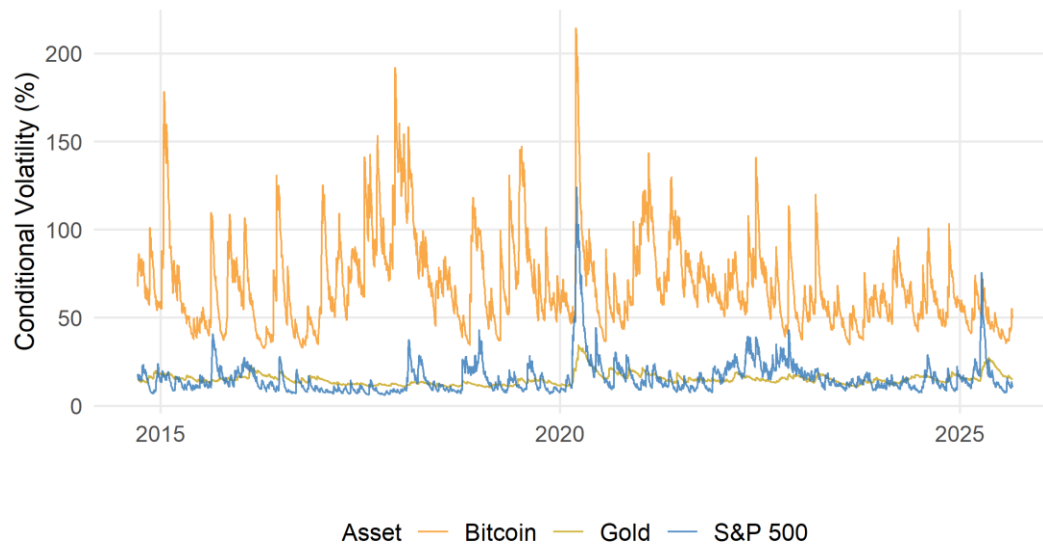
garch_vol_data <- data.frame(
  Date = rep(df_full$Date, 3),
  Volatility = c(df_full$Gold_GARCH_Vol, df_full$SP500_GARCH_Vol,
df_full$BTC_GARCH_Vol),
  Asset = factor(rep(c("Gold", "S&P 500", "Bitcoin"), each = nrow(df_full)),
    levels = c("Bitcoin", "Gold", "S&P 500"))
)

ggplot(garch_vol_data, aes(x = Date, y = Volatility, color = Asset)) +
  geom_line(linewidth = 0.5, alpha = 0.8) +
  scale_color_manual(values = c("Bitcoin" = "#F7931A", "Gold" = "#C8A415",
"S&P 500" = "#2E75B6")) +
  labs(title = "GARCH(1,1) Conditional Volatility (Annualized)",
    subtitle = "Bitcoin vs gold vs S&P 500, Student-t distribution",
    y = "Conditional Volatility (%)", x = "") +
  theme_thesis() +
  theme(legend.position = "bottom",
    plot.title = element_text(face = "bold"))

```

GARCH(1,1) Conditional Volatility (Annualized)

Bitcoin vs gold vs S&P 500, Student-t distribution



```
garch_vol_summary <- data.frame(  
  Asset = c("Gold", "S&P 500", "Bitcoin"),  
  Mean = round(c(mean(df_full$Gold_GARCH_Vol),  
    mean(df_full$SP500_GARCH_Vol), mean(df_full$BTC_GARCH_Vol)), 2),  
  Median = round(c(median(df_full$Gold_GARCH_Vol),  
    median(df_full$SP500_GARCH_Vol), median(df_full$BTC_GARCH_Vol)), 2),  
  Min = round(c(min(df_full$Gold_GARCH_Vol), min(df_full$SP500_GARCH_Vol),  
    min(df_full$BTC_GARCH_Vol)), 2),  
  Max = round(c(max(df_full$Gold_GARCH_Vol), max(df_full$SP500_GARCH_Vol),  
    max(df_full$BTC_GARCH_Vol)), 2)  
)
```

```
kable(garch_vol_summary, align = "lcccc")
```

Asset	Mean	Median	Min	Max
Gold	15.08	14.61	10.19	34.38
S&P 500	15.80	13.26	6.00	124.09
Bitcoin	69.09	63.24	32.74	214.23

```
cat("=== Gold GARCH Residual Tests ===\n")
```

```
## === Gold GARCH Residual Tests ===
```

```
gold_std_resid <- as.numeric(residuals(gold_garch) / sigma(gold_garch))
```

```
lb_gold <- Box.test(gold_std_resid, lag = 20, type = "Ljung-Box")
```

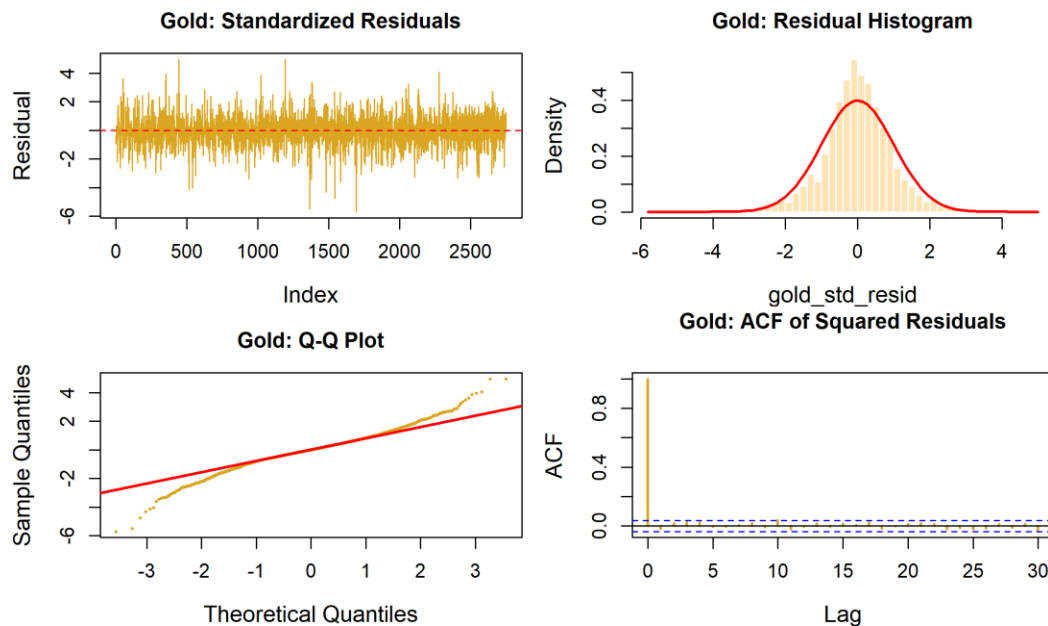
```
cat(sprintf("Ljung-Box (residuals): stat=%.4f, p=%.4f\n", lb_gold$statistic,  
  lb_gold$p.value))
```

```
## Ljung-Box (residuals): stat=19.4792, p=0.4909
```

```
lb_gold_sq <- Box.test(gold_std_resid^2, lag = 20, type = "Ljung-Box")
cat(sprintf("Ljung-Box (squared): stat=%.4f, p=%.4f\n",
lb_gold_sq$statistic, lb_gold_sq$p.value))

## Ljung-Box (squared): stat=17.5875, p=0.6146

par(mfrow = c(2, 2), mar = c(4, 4, 3, 1), cex.lab = 1.25, cex.axis = 1.05,
cex.main = 1.15)
plot(gold_std_resid, type = "l", main = "Gold: Standardized Residuals", ylab =
"Residual", col = "#DAA520")
abline(h = 0, col = "red", lty = 2)
hist(gold_std_resid, breaks = 50, prob = TRUE, main = "Gold: Residual
Histogram",
col = "#FFE4B5", border = "white")
curve(dnorm(x, 0, 1), add = TRUE, col = "red", lwd = 2)
qqnorm(gold_std_resid, main = "Gold: Q-Q Plot", col = "#DAA520", pch = 16,
cex = 0.4)
qqline(gold_std_resid, col = "red", lwd = 2)
acf(gold_std_resid^2, lag.max = 30, main = "Gold: ACF of Squared Residuals",
col = "#DAA520", lwd = 2)
```



```
cat("=== Bitcoin GARCH Residual Tests ===\n")

## === Bitcoin GARCH Residual Tests ===

btc_std_resid <- as.numeric(residuals(btc_garch) / sigma(btc_garch))

lb_btc <- Box.test(btc_std_resid, lag = 20, type = "Ljung-Box")
cat(sprintf("Ljung-Box (residuals): stat=%.4f, p=%.4f\n", lb_btc$statistic,
lb_btc$p.value))

## Ljung-Box (residuals): stat=38.0903, p=0.0086
```

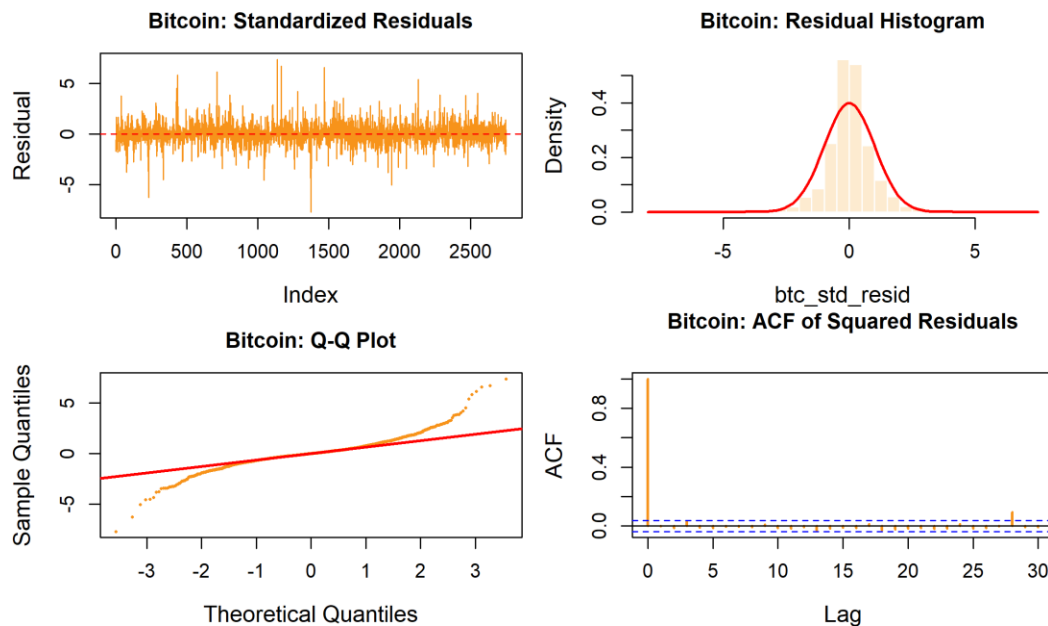
```

lb_btc_sq <- Box.test(btc_std_resid^2, lag = 20, type = "Ljung-Box")
cat(sprintf("Ljung-Box (squared):  stat=%.4f, p=%.4f\n",
lb_btc_sq$statistic, lb_btc_sq$p.value))

## Ljung-Box (squared):  stat=17.0799, p=0.6478

par(mfrow = c(2, 2), mar = c(4, 4, 3, 1), cex.lab = 1.25, cex.axis = 1.05,
cex.main = 1.15)
plot(btc_std_resid, type = "l", main = "Bitcoin: Standardized Residuals",
ylab = "Residual", col = "#F7931A")
abline(h = 0, col = "red", lty = 2)
hist(btc_std_resid, breaks = 50, prob = TRUE, main = "Bitcoin: Residual
Histogram",
col = "#FDEBD0", border = "white")
curve(dnorm(x, 0, 1), add = TRUE, col = "red", lwd = 2)
qqnorm(btc_std_resid, main = "Bitcoin: Q-Q Plot", col = "#F7931A", pch = 16,
cex = 0.4)
qqline(btc_std_resid, col = "red", lwd = 2)
acf(btc_std_resid^2, lag.max = 30, main = "Bitcoin: ACF of Squared
Residuals", col = "#F7931A", lwd = 2)

```



```

cat("=== S&P 500 GARCH Residual Tests ===\n")

## === S&P 500 GARCH Residual Tests ===

sp_std_resid <- as.numeric(residuals(sp_garch) / sigma(sp_garch))

lb_sp <- Box.test(sp_std_resid, lag = 20, type = "Ljung-Box")
cat(sprintf("Ljung-Box (residuals): stat=%.4f, p=%.4f\n", lb_sp$statistic,
lb_sp$p.value))

## Ljung-Box (residuals): stat=22.0528, p=0.3377

```

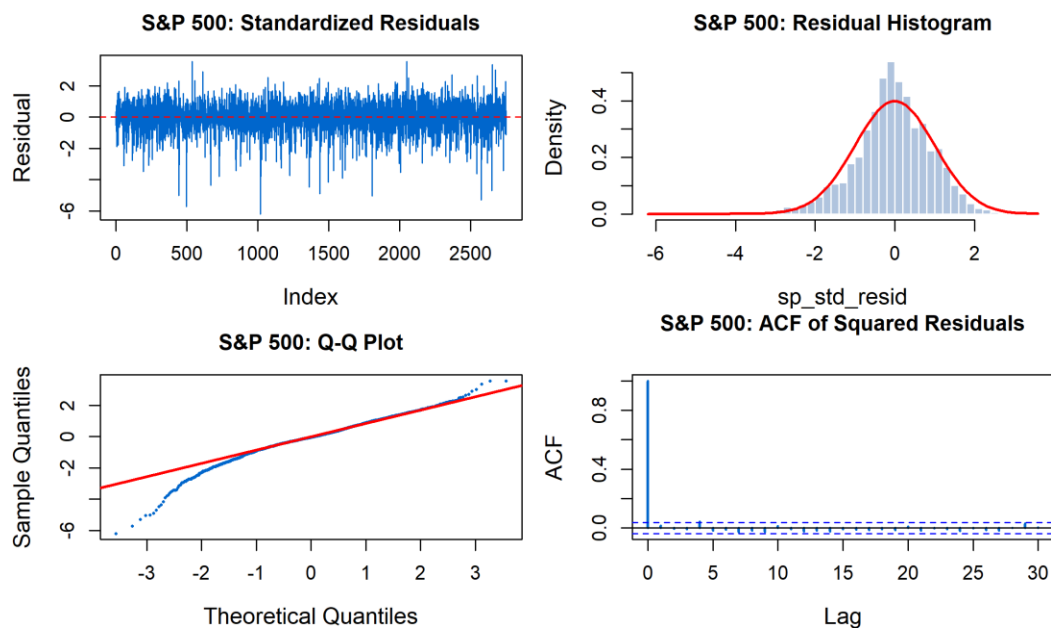
```

lb_sp_sq <- Box.test(sp_std_resid^2, lag = 20, type = "Ljung-Box")
cat(sprintf("Ljung-Box (squared):    stat=%.4f, p=%.4f\n", lb_sp_sq$statistic,
lb_sp_sq$p.value))

## Ljung-Box (squared):    stat=18.8682, p=0.5304

par(mfrow = c(2, 2), mar = c(4, 4, 3, 1), cex.lab = 1.25, cex.axis = 1.05,
cex.main = 1.15)
plot(sp_std_resid, type = "l", main = "S&P 500: Standardized Residuals", ylab =
"Residual", col = "#0066CC")
abline(h = 0, col = "red", lty = 2)
hist(sp_std_resid, breaks = 50, prob = TRUE, main = "S&P 500: Residual
Histogram",
col = "#B0C4DE", border = "white")
curve(dnorm(x, 0, 1), add = TRUE, col = "red", lwd = 2)
qqnorm(sp_std_resid, main = "S&P 500: Q-Q Plot", col = "#0066CC", pch = 16,
cex = 0.4)
qqline(sp_std_resid, col = "red", lwd = 2)
acf(sp_std_resid^2, lag.max = 30, main = "S&P 500: ACF of Squared Residuals",
col = "#0066CC", lwd = 2)

```



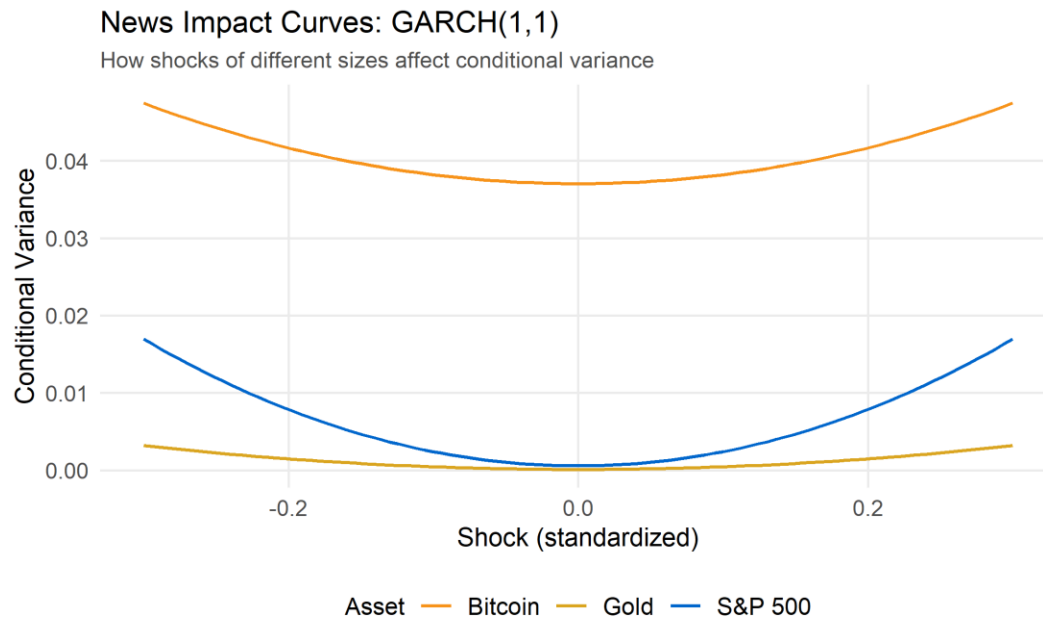
```

ni_gold <- newsimpact(gold_garch)
ni_btc <- newsimpact(btc_garch)
ni_sp <- newsimpact(sp_garch)

ni_df <- data.frame(
  Shock = c(ni_gold$zx, ni_btc$zx, ni_sp$zx),
  Variance = c(ni_gold$zy, ni_btc$zy, ni_sp$zy),
  Asset = rep(c("Gold", "Bitcoin", "S&P 500"), each = length(ni_gold$zx))
)

```

```
ggplot(ni_df, aes(x = Shock, y = Variance, color = Asset)) +
  geom_line(linewidth = 0.8) +
  scale_color_manual(values = c("Bitcoin" = "#F7931A", "Gold" = "#DAA520",
"S&P 500" = "#0066CC")) +
  labs(title = "News Impact Curves: GARCH(1,1)",
        subtitle = "How shocks of different sizes affect conditional
variance",
        x = "Shock (standardized)", y = "Conditional Variance") +
  theme_thesis() +
  theme(legend.position = "bottom")
```



```
# Prepare return matrix (Gold, Bitcoin, SP500)
returns_mat <- df_full[, c("Gold_Return", "BTC_Return", "SP500_Return")]
returns_mat <- returns_mat[complete.cases(returns_mat), ]

# Step 1: Univariate GARCH specs for each series
uspec <- multispec(replicate(3, ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model      = list(armaOrder = c(0, 0), include.mean = TRUE),
  distribution.model = "std"
)))

# Step 2: DCC specification
dcc_spec <- dccspec(uspec = uspec, dccOrder = c(1, 1), distribution = "mvt")

# Step 3: Fit DCC model
dcc_fit <- dccfit(dcc_spec, data = returns_mat)
cat("=== DCC-GARCH Model ===\n")

## === DCC-GARCH Model ===
```

```
print(dcc_fit)
```

```
##
## *-----*
## *          DCC GARCH Fit          *
## *-----*
##
## Distribution      : mvt
## Model            : DCC(1,1)
## No. Parameters    : 21
## [VAR GARCH DCC UncQ] : [0+15+3+3]
## No. Series        : 3
## No. Obs.          : 2753
## Log-Likelihood    : 23660.76
## Av.Log-Likelihood : 8.59
##
## Optimal Parameters
## -----
##              Estimate   Std. Error   t value   Pr(>|t|)
## [Gold_Return].mu      0.000454    0.000149    3.04548 0.002323
## [Gold_Return].omega    0.000001    0.000003    0.32912 0.742065
## [Gold_Return].alpha1   0.034210    0.020042    1.70687 0.087846
## [Gold_Return].beta1    0.956600    0.014402   66.42275 0.000000
## [Gold_Return].shape    4.442618    1.872158    2.37299 0.017645
## [BTC_Return].mu        0.001600    0.000472    3.38968 0.000700
## [BTC_Return].omega     0.000042    0.000019    2.19548 0.028129
## [BTC_Return].alpha1    0.115510    0.018753    6.15943 0.000000
## [BTC_Return].beta1     0.883490    0.023026   38.36868 0.000000
## [BTC_Return].shape     3.064182    0.123186   24.87439 0.000000
## [SP500_Return].mu      0.000940    0.000123    7.64860 0.000000
## [SP500_Return].omega    0.000002    0.000002    1.37944 0.167759
## [SP500_Return].alpha1  0.181916    0.036068    5.04370 0.000000
## [SP500_Return].beta1   0.814923    0.030498   26.72012 0.000000
## [SP500_Return].shape    5.560640    0.631308    8.80813 0.000000
## [Joint]dcca1           0.017593    0.004452    3.95121 0.000078
## [Joint]dccb1           0.971252    0.008856  109.67311 0.000000
## [Joint]mshape          4.885219    0.228347   21.39385 0.000000
##
## Information Criteria
## -----
##
## Akaike      -17.174
## Bayes       -17.129
## Shibata     -17.174
## Hannan-Quinn -17.158
##
##
## Elapsed time : 6.033365
```

```

dcc_coefs <- coef(dcc_fit)

cat("\nDCC Parameters:\n")

##
## DCC Parameters:

cat(sprintf(" dcca1 (news):          %.4f\n", dcc_coefs["[Joint]dcca1"]))
## dcca1 (news):          0.0176

cat(sprintf(" dccb1 (persistence): %.4f\n", dcc_coefs["[Joint]dccb1"]))
## dccb1 (persistence): 0.9713

cat(sprintf(" Sum (persistence):   %.4f\n",
            dcc_coefs["[Joint]dcca1"] + dcc_coefs["[Joint]dccb1"]))
## Sum (persistence):   0.9888

cat("\nHigher dcca1 = correlations react more to shocks")

##
## Higher dcca1 = correlations react more to shocks

cat("\nHigher dccb1 = correlations are more persistent")

##
## Higher dccb1 = correlations are more persistent

cat("\nSum close to 1 = correlation changes are very long-lasting\n")

##
## Sum close to 1 = correlation changes are very long-lasting

# Extract time-varying correlations
dcc_cor <- rcor(dcc_fit)

# Gold <-> SP500: [1,3] | Bitcoin <-> SP500: [2,3] | Gold <-> Bitcoin: [1,2]
n_obs <- dim(dcc_cor)[3]
dcc_dates <- tail(df_full$Date, n_obs)

gold_sp_dcc <- dcc_cor[1, 3, ]
btc_sp_dcc <- dcc_cor[2, 3, ]
gold_btc_dcc <- dcc_cor[1, 2, ]

dcc_plot_data <- data.frame(
  Date = rep(dcc_dates, 3),
  Correlation = c(gold_sp_dcc, btc_sp_dcc, gold_btc_dcc),
  Pair = factor(rep(c("Gold <-> S&P 500", "Bitcoin <-> S&P 500", "Gold <->
Bitcoin"),
                    each = n_obs))

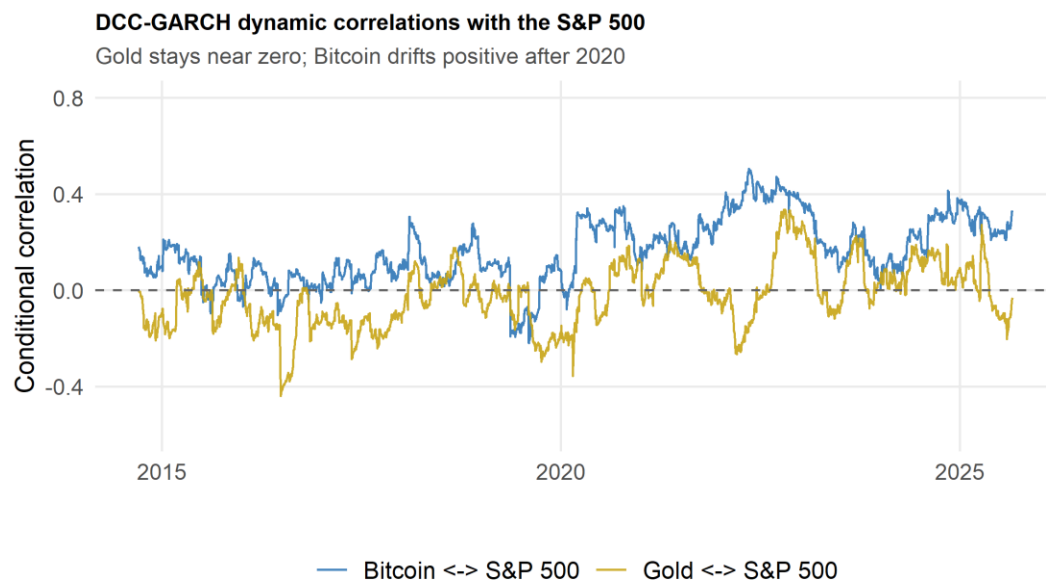
```

```

)

ggplot(dcc_plot_data[dcc_plot_data$Pair != "Gold <-> Bitcoin", ],
      aes(x = Date, y = Correlation, color = Pair, linetype = Pair)) +
  geom_line(linewidth = 0.6, alpha = 0.9) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  scale_color_manual(values = c("Gold <-> S&P 500" = "#C8A415",
                                "Bitcoin <-> S&P 500" = "#2E75B6")) +
  scale_linetype_manual(values = c("Gold <-> S&P 500" = "solid",
                                   "Bitcoin <-> S&P 500" = "solid")) +
  labs(title = "DCC-GARCH dynamic correlations with the S&P 500",
       subtitle = "Gold stays near zero; Bitcoin drifts positive after 2020",
       y = "Conditional correlation", x = "", color = NULL, linetype = NULL)
+
ylim(-0.6, 0.8) +
theme_thesis() +
theme(legend.position = "bottom",
      plot.title = element_text(size = 12, face = "bold"))

```



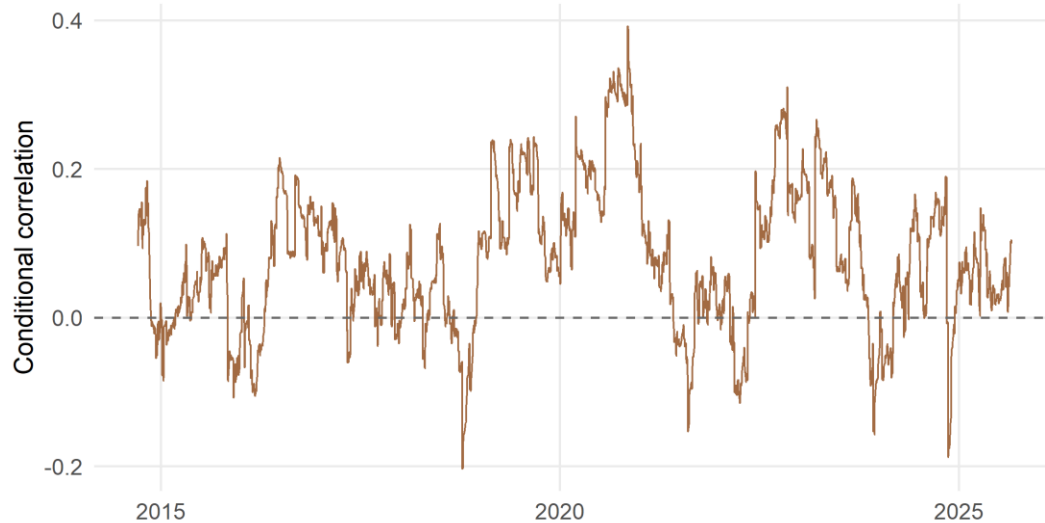
```

ggplot(dcc_plot_data[dcc_plot_data$Pair == "Gold <-> Bitcoin", ],
      aes(x = Date, y = Correlation)) +
  geom_line(linewidth = 0.5, color = "#8B4513", alpha = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  labs(title = "DCC-GARCH: Gold-Bitcoin Dynamic Correlation",
       subtitle = "Small and positive, not the near-unity of 'digital gold'",
       y = "Conditional correlation", x = "") +
  theme_thesis() +
  theme(plot.title = element_text(face = "bold"))

```

DCC-GARCH: Gold-Bitcoin Dynamic Correlation

Small and positive, not the near-unity of 'digital gold'



```

dcc_summary <- data.frame(
  Pair = c("Gold <-> S&P 500", "Bitcoin <-> S&P 500", "Gold <-> Bitcoin"),
  Mean  = round(c(mean(gold_sp_dcc), mean(btc_sp_dcc), mean(gold_btc_dcc)),
4),
  Median = round(c(median(gold_sp_dcc), median(btc_sp_dcc),
median(gold_btc_dcc)), 4),
  Min    = round(c(min(gold_sp_dcc), min(btc_sp_dcc), min(gold_btc_dcc)), 4),
  Max    = round(c(max(gold_sp_dcc), max(btc_sp_dcc), max(gold_btc_dcc)), 4),
  `% Time Negative` = round(c(mean(gold_sp_dcc < 0) * 100,
                              mean(btc_sp_dcc < 0) * 100,
                              mean(gold_btc_dcc < 0) * 100), 1),

  check.names = FALSE
)

kable(dcc_summary, align = "lccccc")

```

Pair	Mean	Median	Min	Max	% Time Negative
Gold <-> S&P 500	-0.0217	-0.0168	-0.4384	0.3340	56.6
Bitcoin <-> S&P 500	0.1622	0.1468	-0.2189	0.5038	8.1
Gold <-> Bitcoin	0.0775	0.0714	-0.2037	0.3920	21.0

```

# =====
# DCC mean split across the 2020 structural break [smaller fix]
# =====
# A single full-sample mean DCC correlation averages across the
# regime change documented in the year-by-year results. We report
# the mean DCC correlations separately for the pre-2020 and
# post-2020 sub-periods so the break is not hidden by pooling.

```

```
break_date <- as.Date("2020-01-01")
```

```

pre_idx    <- dcc_dates < break_date
post_idx   <- dcc_dates >= break_date

dcc_break_tbl <- data.frame(
  Pair = c("Gold <-> S&P 500", "Bitcoin <-> S&P 500", "Gold <-> Bitcoin"),
  Mean_pre2020 = round(c(mean(gold_sp_dcc[pre_idx]),
mean(btc_sp_dcc[pre_idx]), mean(gold_btc_dcc[pre_idx])), 4),
  Mean_post2020 = round(c(mean(gold_sp_dcc[post_idx]),
mean(btc_sp_dcc[post_idx]), mean(gold_btc_dcc[post_idx])), 4),
  N_pre = sum(pre_idx),
  N_post = sum(post_idx),
  stringsAsFactors = FALSE
)
cat("DCC mean correlations split at 2020-01-01:\n")

## DCC mean correlations split at 2020-01-01:

print(dcc_break_tbl)

##           Pair Mean_pre2020 Mean_post2020 N_pre N_post
## 1   Gold <-> S&P 500    -0.0871      0.0393  1328  1425
## 2 Bitcoin <-> S&P 500     0.0660      0.2519  1328  1425
## 3   Gold <-> Bitcoin     0.0603      0.0935  1328  1425

# Static return-correlation matrix (computed once; the original
# duplicated this block).
cor_matrix_ret <- cor(df_full[, c("Gold_Return", "BTC_Return",
"SP500_Return")], use = "complete.obs")
colnames(cor_matrix_ret) <- c("Gold", "Bitcoin", "S&P 500")
rownames(cor_matrix_ret) <- c("Gold", "Bitcoin", "S&P 500")
kable(round(cor_matrix_ret, 4), align = "ccc")

```

	Gold	Bitcoin	S&P 500
Gold	1.0000	0.0787	0.0218
Bitcoin	0.0787	1.0000	0.2158
S&P 500	0.0218	0.2158	1.0000

```

df_full$Gold_SP_Corr_90d <- rollapply(
  df_full[, c("Gold_Return", "SP500_Return")],
  width = 90,
  FUN = function(x) cor(x[,1], x[,2], use = "complete.obs"),
  by.column = FALSE, fill = NA, align = "right"
)

df_full$BTC_SP_Corr_90d <- rollapply(
  df_full[, c("BTC_Return", "SP500_Return")],
  width = 90,
  FUN = function(x) cor(x[,1], x[,2], use = "complete.obs"),
  by.column = FALSE, fill = NA, align = "right"
)

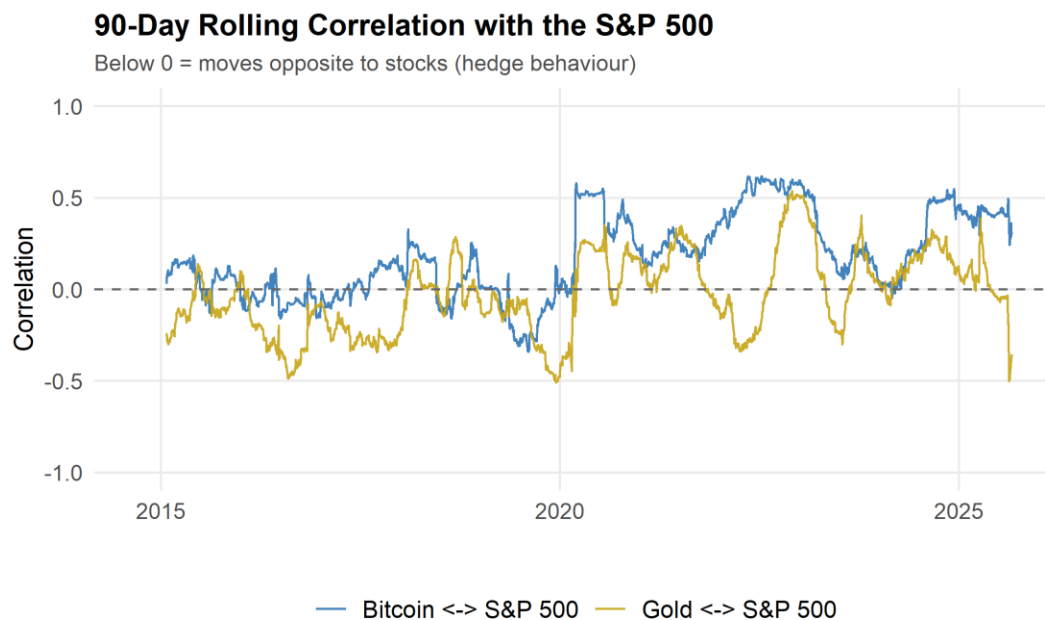
```

```
corr_plot_data <- data.frame(
  Date = rep(df_full$Date, 2),
  Correlation = c(df_full$Gold_SP_Corr_90d, df_full$BTC_SP_Corr_90d),
  Asset = factor(rep(c("Gold <-> S&P 500", "Bitcoin <-> S&P 500"), each =
nrow(df_full)))
)
```

```
ggplot(corr_plot_data, aes(x = Date, y = Correlation, color = Asset)) +
  geom_line(linewidth = 0.6, alpha = 0.9) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  scale_color_manual(values = c("Gold <-> S&P 500" = "#C8A415",
                                "Bitcoin <-> S&P 500" = "#2E75B6")) +
  labs(title = "90-Day Rolling Correlation with the S&P 500",
        subtitle = "Below 0 = moves opposite to stocks (hedge behaviour)",
        y = "Correlation", x = "", color = NULL) +
  ylim(-1, 1) +
  theme_thesis() +
  theme(legend.position = "bottom",
        plot.title = element_text(face = "bold"))
```

```
## Warning: Removed 178 rows containing missing values or values outside the
scale range
```

```
## (`geom_line()`).
```



```
corr_summary <- data.frame(
  Pair = c("Gold <-> S&P 500", "Bitcoin <-> S&P 500"),
  Mean = round(c(mean(df_full$Gold_SP_Corr_90d, na.rm = TRUE),
                  mean(df_full$BTC_SP_Corr_90d, na.rm = TRUE)), 4),
  Median = round(c(median(df_full$Gold_SP_Corr_90d, na.rm = TRUE),
                   median(df_full$BTC_SP_Corr_90d, na.rm = TRUE)), 4),
```

```

Min    = round(c(min(df_full$Gold_SP_Corr_90d, na.rm = TRUE),
                  min(df_full$BTC_SP_Corr_90d, na.rm = TRUE)), 4),
Max    = round(c(max(df_full$Gold_SP_Corr_90d, na.rm = TRUE),
                  max(df_full$BTC_SP_Corr_90d, na.rm = TRUE)), 4),
`% Time Negative` = round(c(mean(df_full$Gold_SP_Corr_90d < 0, na.rm =
TRUE) * 100,
                           mean(df_full$BTC_SP_Corr_90d < 0, na.rm =
TRUE) * 100), 1),
  check.names = FALSE
)

kable(corr_summary, align = "lcccc")

```

Pair	Mean	Median	Min	Max	% Time Negative
Gold <-> S&P 500	-0.0297	-0.0389	-0.5081	0.5350	55.8
Bitcoin <-> S&P 500	0.1783	0.1548	-0.3425	0.6193	23.6

```

# =====
# VIX regime split using the OFFICIAL CBOE VIX [fix #1]
# =====
# The thesis uses the official, option-implied CBOE VIX to define
# stress vs calm regimes -- NOT a rolling-volatility proxy.
#
# Following the quantile-based safe-haven literature (a safe haven
# is judged by behaviour in the EXTREME tail of the stress
# distribution, not merely above-average stress), we define the
# high-stress regime as days on which the VIX exceeds its 75th
# sample percentile (top quartile). Days at or below the 75th
# percentile form the calm/low-stress regime. We also construct a
# stricter top-decile (90th percentile) cut as a robustness check
# on the most extreme stress days.

stopifnot("VIX_CBOE" %in% colnames(df_full))

vix_q75 <- quantile(df_full$VIX_CBOE, 0.75, na.rm = TRUE)
vix_q90 <- quantile(df_full$VIX_CBOE, 0.90, na.rm = TRUE)

# Primary split: top quartile = stress
df_full$VIX_regime <- ifelse(df_full$VIX_CBOE > vix_q75,
                           "High VIX (Stress)", "Low VIX (Calm)")
# Robustness split: top decile = extreme stress
df_full$VIX_regime_p90 <- ifelse(df_full$VIX_CBOE > vix_q90,
                                "Extreme VIX (Top 10%)", "Rest")

regimes <- df_full[!is.na(df_full$VIX_regime) & !is.na(df_full$VIX_CBOE), ]

cat(sprintf("VIX 75th percentile (primary stress cut): %.2f\n", vix_q75))
## VIX 75th percentile (primary stress cut): 21.25

```

```

cat(sprintf("VIX 90th percentile (robustness cut):      %.2f\n", vix_q90))
## VIX 90th percentile (robustness cut):      26.90

cat("Observations by primary regime (top quartile = stress):\n")
## Observations by primary regime (top quartile = stress):

print(table(regimes$VIX_regime))

##
## High VIX (Stress)      Low VIX (Calm)
##           688           2065

cat("\nObservations by robustness regime (top decile = extreme):\n")
##
## Observations by robustness regime (top decile = extreme):

print(table(regimes$VIX_regime_p90))

##
## Extreme VIX (Top 10%)      Rest
##           276           2477

regime_ret <- do.call(rbind, lapply(split(regimes, regimes$VIX_regime),
function(sub) {
  data.frame(
    Regime = sub$VIX_regime[1],
    `Gold Ann. Return (%)` = round(mean(sub$Gold_Return, na.rm = TRUE) * 252
* 100, 2),
    `Bitcoin Ann. Return (%)` = round(mean(sub$BTC_Return, na.rm = TRUE) *
252 * 100, 2),
    `S&P 500 Ann. Return (%)` = round(mean(sub$SP500_Return, na.rm = TRUE) *
252 * 100, 2),
    check.names = FALSE
  )
}))
rownames(regime_ret) <- NULL
kable(regime_ret, align = "lccc")

```

Regime	Gold Ann. Return (%)	Bitcoin Ann. Return (%)	S&P 500 Ann. Return (%)
High VIX (Stress)	19.85	41.0	-37.23
Low VIX (Calm)	7.53	83.6	28.87

```

regime_vol <- do.call(rbind, lapply(split(regimes, regimes$VIX_regime),
function(sub) {
  data.frame(
    Regime = sub$VIX_regime[1],
    `Gold Vol (%)` = round(sd(sub$Gold_Return, na.rm = TRUE) * sqrt(252) *

```

```

100, 2),
  `Bitcoin Vol (%)` = round(sd(sub$BTC_Return, na.rm = TRUE) * sqrt(252) *
100, 2),
  `S&P 500 Vol (%)` = round(sd(sub$SP500_Return, na.rm = TRUE) * sqrt(252)
* 100, 2),
  check.names = FALSE
)
}))
rownames(regime_vol) <- NULL
kable(regime_vol, align = "lccc")

```

Regime	Gold Vol (%)	Bitcoin Vol (%)	S&P 500 Vol (%)
High VIX (Stress)	19.52	75.81	29.97
Low VIX (Calm)	13.25	64.41	11.30

```

regime_corr <- do.call(rbind, lapply(split(regimes, regimes$VIX_regime),
function(sub) {
  data.frame(
    Regime = sub$VIX_regime[1],
    `Gold <-> S&P 500` = round(cor(sub$Gold_Return, sub$SP500_Return, use =
"complete.obs"), 4),
    `Bitcoin <-> S&P 500` = round(cor(sub$BTC_Return, sub$SP500_Return, use =
"complete.obs"), 4),
    check.names = FALSE
  )
}))
rownames(regime_corr) <- NULL
kable(regime_corr, align = "lcc")

```

Regime	Gold <-> S&P 500	Bitcoin <-> S&P 500
High VIX (Stress)	0.0728	0.3931
Low VIX (Calm)	-0.0367	0.0670

```

# Robustness: correlation with the S&P 500 on the most extreme
# stress days (VIX in its top decile) vs the rest of the sample.
regime_corr_p90 <- do.call(rbind, lapply(split(regimes,
regimes$VIX_regime_p90), function(sub) {
  data.frame(
    Regime = sub$VIX_regime_p90[1],
    `Gold <-> S&P 500` = round(cor(sub$Gold_Return, sub$SP500_Return, use =
"complete.obs"), 4),
    `Bitcoin <-> S&P 500` = round(cor(sub$BTC_Return, sub$SP500_Return, use =
"complete.obs"), 4),
    check.names = FALSE
  )
}))
rownames(regime_corr_p90) <- NULL
kable(regime_corr_p90, align = "lcc")

```

Regime	Gold <-> S&P 500	Bitcoin <-> S&P 500
--------	------------------	---------------------

Regime	Gold <-> S&P 500	Bitcoin <-> S&P 500
--------	------------------	---------------------

Extreme VIX (Top 10%)	0.1628	0.4882
-----------------------	--------	--------

Rest	-0.0541	0.1104
------	---------	--------

```
garch_regime <- do.call(rbind, lapply(split(regimes, regimes$VIX_regime),
function(sub) {
  data.frame(
    Regime = sub$VIX_regime[1],
    `Gold GARCH Vol (%)` = round(mean(sub$Gold_GARCH_Vol, na.rm = TRUE), 2),
    `Bitcoin GARCH Vol (%)` = round(mean(sub$BTC_GARCH_Vol, na.rm = TRUE),
2),
    `S&P 500 GARCH Vol (%)` = round(mean(sub$SP500_GARCH_Vol, na.rm = TRUE),
2),
    check.names = FALSE
  )
}))
rownames(garch_regime) <- NULL
kable(garch_regime, align = "lccc")
```

Regime	Gold GARCH Vol (%)	Bitcoin GARCH Vol (%)	S&P 500 GARCH Vol (%)
High VIX (Stress)	17.30	74.42	25.51
Low VIX (Calm)	14.35	67.32	12.57

```
df_full$Year <- format(df_full$Date, "%Y")
```

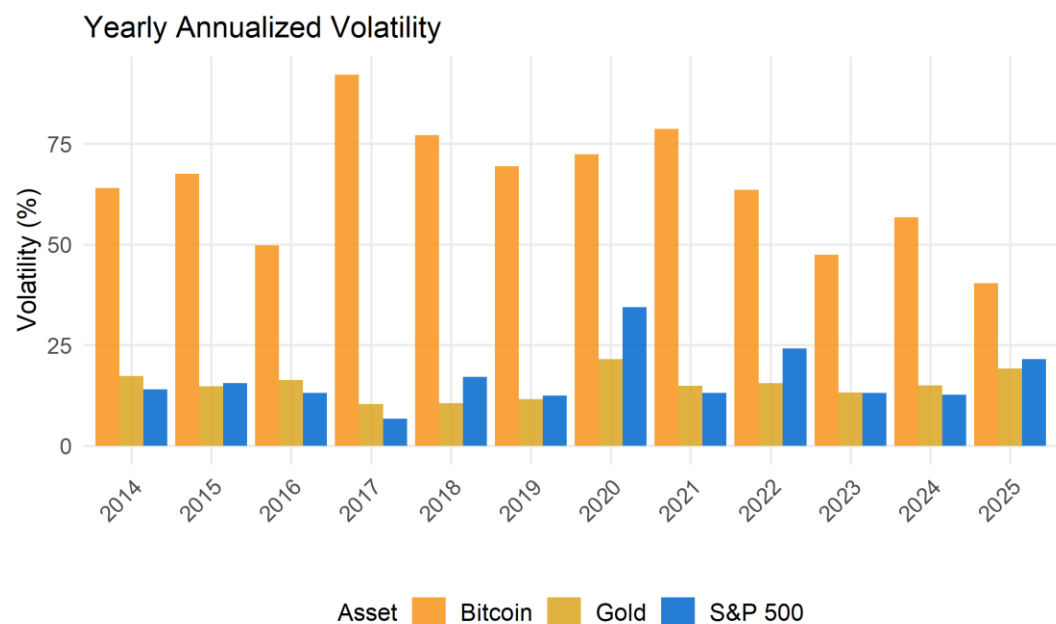
```
yearly <- do.call(rbind, lapply(split(df_full, df_full$Year), function(yr) {
  data.frame(
    Year = yr$Year[1],
    Gold_Vol = round(sd(yr$Gold_Return, na.rm = TRUE) * sqrt(252) * 100,
2),
    BTC_Vol = round(sd(yr$BTC_Return, na.rm = TRUE) * sqrt(252) * 100,
2),
    SP500_Vol = round(sd(yr$SP500_Return, na.rm = TRUE) * sqrt(252) *
100, 2),
    Gold_SP_Corr = round(cor(yr$Gold_Return, yr$SP500_Return, use =
"complete.obs"), 4),
    BTC_SP_Corr = round(cor(yr$BTC_Return, yr$SP500_Return, use =
"complete.obs"), 4),
    Gold_BTC_Corr = round(cor(yr$Gold_Return, yr$BTC_Return, use =
"complete.obs"), 4)
  )
}))
rownames(yearly) <- NULL

kable(yearly[, c("Year", "Gold_Vol", "BTC_Vol", "SP500_Vol")],
      col.names = c("Year", "Gold (%)", "Bitcoin (%)", "S&P 500 (%)"),
      align = "lccc")
```

Year	Gold (%)	Bitcoin (%)	S&P 500 (%)
2014	17.28	63.99	14.01
2015	14.73	67.46	15.49
2016	16.27	49.70	13.10
2017	10.39	92.12	6.69
2018	10.54	77.06	17.13
2019	11.53	69.42	12.47
2020	21.44	72.34	34.43
2021	14.91	78.60	13.10
2022	15.51	63.56	24.17
2023	13.24	47.38	13.09
2024	14.97	56.74	12.65
2025	19.13	40.35	21.46

```
yearly_vol_long <- melt(yearly[, c("Year", "Gold_Vol", "BTC_Vol",
"SP500_Vol")],
                        id.vars = "Year", variable.name = "Asset", value.name
= "Volatility")
yearly_vol_long$Asset <- gsub("Gold_Vol", "Gold", yearly_vol_long$Asset)
yearly_vol_long$Asset <- gsub("BTC_Vol", "Bitcoin", yearly_vol_long$Asset)
yearly_vol_long$Asset <- gsub("SP500_Vol", "S&P 500", yearly_vol_long$Asset)

ggplot(yearly_vol_long, aes(x = Year, y = Volatility, fill = Asset)) +
  geom_bar(stat = "identity", position = "dodge", alpha = 0.85) +
  scale_fill_manual(values = c("Bitcoin" = "#F7931A", "Gold" = "#DAA520",
"S&P 500" = "#0066CC")) +
  labs(title = "Yearly Annualized Volatility", y = "Volatility (%)", x = "")
+
  theme_thesis() +
  theme(legend.position = "bottom", axis.text.x = element_text(angle = 45,
hjust = 1))
```

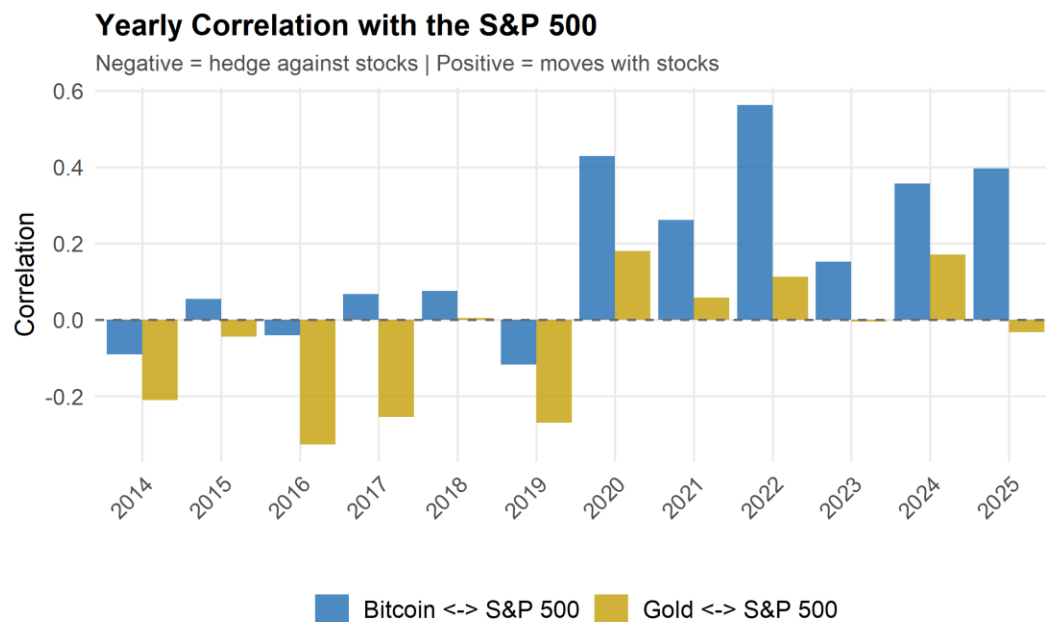


```
kable(yearly[, c("Year", "Gold_SP_Corr", "BTC_SP_Corr", "Gold_BTC_Corr")],
      col.names = c("Year", "Gold <-> S&P", "Bitcoin <-> S&P", "Gold <->
Bitcoin"),
      align = "lccc")
```

Year	Gold <-> S&P	Bitcoin <-> S&P	Gold <-> Bitcoin
2014	-0.2107	-0.0910	-0.0602
2015	-0.0440	0.0547	-0.0073
2016	-0.3270	-0.0411	0.1094
2017	-0.2550	0.0670	-0.0113
2018	0.0044	0.0750	0.0124
2019	-0.2703	-0.1179	0.2154
2020	0.1802	0.4289	0.2774
2021	0.0582	0.2616	-0.0512
2022	0.1132	0.5632	0.0991
2023	-0.0050	0.1521	0.1118
2024	0.1706	0.3566	0.0643
2025	-0.0325	0.3964	0.0441

```
yearly_corr_long <- melt(yearly[, c("Year", "Gold_SP_Corr", "BTC_SP_Corr")],
                        id.vars = "Year", variable.name = "Pair", value.name
= "Correlation")
yearly_corr_long$Pair <- gsub("Gold_SP_Corr", "Gold <-> S&P 500",
yearly_corr_long$Pair)
yearly_corr_long$Pair <- gsub("BTC_SP_Corr", "Bitcoin <-> S&P 500",
yearly_corr_long$Pair)
```

```
ggplot(yearly_corr_long, aes(x = Year, y = Correlation, fill = Pair)) +
  geom_bar(stat = "identity", position = "dodge", alpha = 0.85) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  scale_fill_manual(values = c("Gold <-> S&P 500" = "#C8A415",
                              "Bitcoin <-> S&P 500" = "#2E75B6")) +
  labs(title = "Yearly Correlation with the S&P 500",
       subtitle = "Negative = hedge against stocks | Positive = moves with stocks",
       y = "Correlation", x = "", fill = NULL) +
  theme_thesis() +
  theme(legend.position = "bottom",
        plot.title = element_text(face = "bold"),
        axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# =====
# Equal-volatility test: robust version [smaller fix]
# =====
# var.test (an F-test) assumes normality, which daily returns
# violate (fat tails). We report it for continuity but rely on
# Levene's test (Brown-Forsyth, center = median), which is robust
# to non-normality, for the actual inference.
```

```
f_test <- var.test(df_full$Gold_Return, df_full$BTC_Return)
```

```
lev_df <- data.frame(
  ret = c(df_full$Gold_Return, df_full$BTC_Return),
  asset = factor(rep(c("Gold", "Bitcoin"),
                    c(length(df_full$Gold_Return),
                      length(df_full$BTC_Return))))
```

```

)
lev_df <- lev_df[is.finite(lev_df$ret), ]
lev_test <- car::leveneTest(ret ~ asset, data = lev_df, center = median)

f_result <- data.frame(
  Statistic = c("F-statistic (assumes normality)", "F-test p-value",
    "Levene (robust) statistic", "Levene p-value",
    "Gold Variance", "Bitcoin Variance", "Result (robust)"),
  Value = c(
    round(f_test$statistic, 4),
    formatC(f_test$p.value, format = "e", digits = 2),
    round(lev_test`F value`[1], 4),
    formatC(lev_test`Pr(>F)`[1], format = "e", digits = 2),
    formatC(var(df_full$Gold_Return, na.rm = TRUE), format = "e", digits =
4),
    formatC(var(df_full$BTC_Return, na.rm = TRUE), format = "e", digits = 4),
    ifelse(lev_test`Pr(>F)`[1] < 0.05,
      "Reject H0: volatilities significantly different",
      "Fail to reject H0")
  )
)

kable(f_result, align = "lc")

```

Statistic	Value
F-statistic (assumes normality)	0.0499
F-test p-value	0.00e+00
Levene (robust) statistic	1180.7405
Levene p-value	1.32e-234
Gold Variance	9.0070e-05
Bitcoin Variance	1.8046e-03
Result (robust)	Reject H0: volatilities significantly different

```

n <- sum(complete.cases(df_full[, c("Gold_Return", "BTC_Return",
"SP500_Return"))))
r1 <- cor(df_full$Gold_Return, df_full$SP500_Return, use = "complete.obs")
r2 <- cor(df_full$BTC_Return, df_full$SP500_Return, use = "complete.obs")

z1 <- 0.5 * log((1 + r1) / (1 - r1))
z2 <- 0.5 * log((1 + r2) / (1 - r2))
z_stat <- (z1 - z2) / sqrt(2 / (n - 3))
p_val <- 2 * (1 - pnorm(abs(z_stat)))

z_result <- data.frame(
  Statistic = c("Gold <-> S&P 500 correlation", "Bitcoin <-> S&P 500
correlation",
    "Z-statistic", "p-value", "Sample size (n)", "Result"),

```

```

Value = c(
  round(r1, 4), round(r2, 4),
  round(z_stat, 4),
  formatC(p_val, format = "e", digits = 2),
  n,
  ifelse(p_val < 0.05, "Reject H0: Significantly different correlations",
"Fail to reject H0")
)
)

kable(z_result, align = "lc")

```

Statistic	Value
Gold <-> S&P 500 correlation	0.0218
Bitcoin <-> S&P 500 correlation	0.2158
Z-statistic	-7.3227
p-value	2.43e-13
Sample size (n)	2753
Result	Reject H0: Significantly different correlations

```

# Summary
# NOTE (fix #1): the stress/calm rows below now use the official
# CBOE VIX regime split defined above.

stress_data <- regimes[regimes$VIX_regime == "High VIX (Stress)", ]
calm_data    <- regimes[regimes$VIX_regime == "Low VIX (Calm)", ]

gold_sp_corr <- cor(df_full$Gold_Return, df_full$SP500_Return, use =
"complete.obs")
btc_sp_corr  <- cor(df_full$BTC_Return,  df_full$SP500_Return, use =
"complete.obs")

summary_table <- data.frame(
  Metric = c(
    "Annualized Volatility (Unconditional)",
    "Mean GARCH Conditional Volatility",
    "GARCH Persistence (alpha + beta)",
    "Overall Correlation with S&P 500",
    "DCC Mean Correlation with S&P 500",
    "Correlation with S&P 500 (High VIX / Stress)",
    "Correlation with S&P 500 (Low VIX / Calm)",
    "% Time Negatively Correlated with S&P 500"
  ),
  Gold = c(
    paste0(round(gold_vol, 2), "%"),
    paste0(round(mean(df_full$Gold_GARCH_Vol), 2), "%"),
    round(coef(gold_garch)["alpha1"] + coef(gold_garch)["beta1"], 4),
    round(gold_sp_corr, 4),

```

```

    round(mean(gold_sp_dcc), 4),
    round(cor(stress_data$Gold_Return, stress_data$SP500_Return, use =
"complete.obs"), 4),
    round(cor(calm_data$Gold_Return, calm_data$SP500_Return, use =
"complete.obs"), 4),
    paste0(round(mean(df_full$Gold_SP_Corr_90d < 0, na.rm = TRUE) * 100, 1),
"%")
),
Bitcoin = c(
    paste0(round(btc_vol, 2), "%"),
    paste0(round(mean(df_full$BTC_GARCH_Vol), 2), "%"),
    round(coef(btc_garch)["alpha1"] + coef(btc_garch)["beta1"], 4),
    round(btc_sp_corr, 4),
    round(mean(btc_sp_dcc), 4),
    round(cor(stress_data$BTC_Return, stress_data$SP500_Return, use =
"complete.obs"), 4),
    round(cor(calm_data$BTC_Return, calm_data$SP500_Return, use =
"complete.obs"), 4),
    paste0(round(mean(df_full$BTC_SP_Corr_90d < 0, na.rm = TRUE) * 100, 1),
"%")
)
)

kable(summary_table, align = "lcc")

```

Metric	Gold	Bitcoin
Annualized Volatility (Unconditional)	15.07%	67.44%
Mean GARCH Conditional Volatility	15.08%	69.09%
GARCH Persistence (alpha + beta)	0.9908	0.999
Overall Correlation with S&P 500	0.0218	0.2158
DCC Mean Correlation with S&P 500	-0.0217	0.1622
Correlation with S&P 500 (High VIX / Stress)	0.0728	0.3931
Correlation with S&P 500 (Low VIX / Calm)	-0.0367	0.067
% Time Negatively Correlated with S&P 500	55.8%	23.6%